L14: Landmark Deep RL Algorithms Part 2

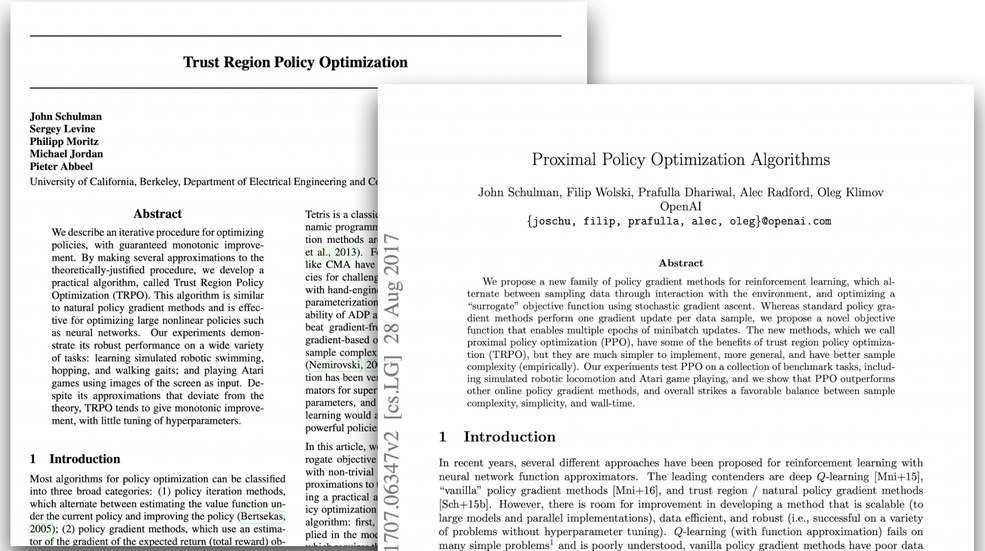
Off-policy methods: DQN, DPG, DDPG, TD3, SAC

EECE 571N | Sequential Decision Making | Fall 2025

Cyrus Neary | cyrus.neary@ubc.ca

Recently we've seen…

TRPO and PPO



Trust Region Policy Optimization

John Schulman
Sergey Levine
Philipp Moritz
Michael Jordan
Pieter Abbeel
University of California, Berkeley, Department of Electrical Engineering and Co

Abstract

We describe an iterative procedure for optimizing policies, with guaranteed monotonic improvement. By making several approximations to the theoretically-justified procedure, we develop a practical algorithm, called Trust Region Policy Optimization (TRPO). This algorithm is similar to natural policy gradient methods and is effective for optimizing large nonlinear policies such as neural networks. Our experiments demonstrate its robust performance on a wide variety of tasks: learning simulated robotic swimming, hopping, and walking gaits; and playing Atari games using images of the screen as input. Despite its approximations that deviate from the theory, TRPO tends to give monotonic improvement, with little tuning of hyperparameters.

1 Introduction

Most algorithms for policy optimization can be classified into three broad categories: (1) policy iteration methods, which alternate between estimating the value function under the current policy and improving the policy (Bertsekas, 2005); (2) policy gradient methods, which use an estima-

Tetris is a classic
namic programm
tion methods an
et al., 2013). Fo
like CMA have
cies for challeng
with hand-engine
parameterization
ability of ADP a
beat gradient-fre
gradient-based o
sample complex
(Nemirovski, 20
tion has been ver
mators for super
parameters, and
learning would a
powerful policie

In this article, w
rogate objective
with non-trivial
proximations to
ing a practical a
icy optimization
algorithm: first,
plied in the mod

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

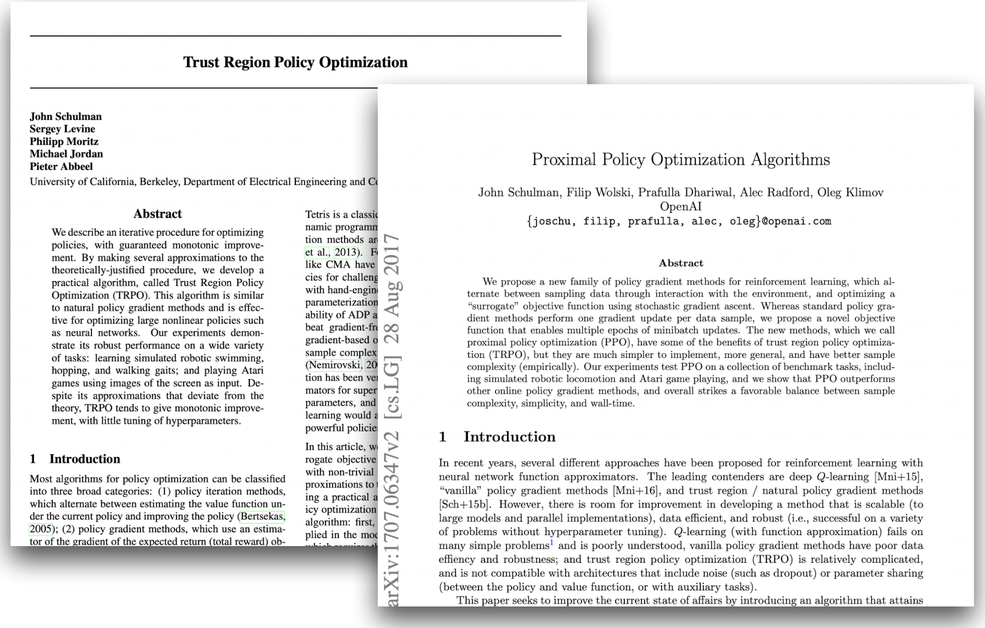
This paper seeks to improve the current state of affairs by introducing an algorithm that attains

arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

- On-policy algorithms.
- Simple (relatively speaking).
- Stable updates.
- Works well empirically, easy to parallelize.
- Works with both discrete and continuous action spaces.

Recently we've seen…

TRPO and PPO



Trust Region Policy Optimization

John Schulman
Sergey Levine
Philipp Moritz
Michael Jordan
Pieter Abbeel

University of California, Berkeley, Department of Electrical Engineering and C

Abstract

We describe an iterative procedure for optimizing policies, with guaranteed monotonic improvement. By making several approximations to the theoretically-justified procedure, we develop a practical algorithm, called Trust Region Policy Optimization (TRPO). This algorithm is similar to natural policy gradient methods and is effective for optimizing large nonlinear policies such as neural networks. Our experiments demonstrate its robust performance on a wide variety of tasks: learning simulated robotic swimming, hopping, and walking gaits; and playing Atari games using images of the screen as input. Despite its approximations that deviate from the theory, TRPO tends to give monotonic improvement, with little tuning of hyperparameters.

1 Introduction

Most algorithms for policy optimization can be classified into three broad categories: (1) policy iteration methods, which alternate between estimating the value function under the current policy and improving the policy (Bertsekas, 2005); (2) policy gradient methods, which use an estima-

Tetris is a classi
namic programm
tion methods an
et al., 2013). F
like CMA have
cies for challeng
with hand-engin
parameterization
ability of ADP a
beat gradient-fr
gradient-based o
sample complex
(Nemirovski, 20
tion has been ve
mators for super
parameters, and
learning would a
powerful policie

In this article, w
rogate objective
with non-trivial
proximations to
ing a practical a
icy optimization
algorithm: first,
plied in the mo

Proximal Policy Optimization Algorithms

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, Oleg Klimov
OpenAI
{joschu, filip, prafulla, alec, oleg}@openai.com

Abstract

We propose a new family of policy gradient methods for reinforcement learning, which alternate between sampling data through interaction with the environment, and optimizing a "surrogate" objective function using stochastic gradient ascent. Whereas standard policy gradient methods perform one gradient update per data sample, we propose a novel objective function that enables multiple epochs of minibatch updates. The new methods, which we call proximal policy optimization (PPO), have some of the benefits of trust region policy optimization (TRPO), but they are much simpler to implement, more general, and have better sample complexity (empirically). Our experiments test PPO on a collection of benchmark tasks, including simulated robotic locomotion and Atari game playing, and we show that PPO outperforms other online policy gradient methods, and overall strikes a favorable balance between sample complexity, simplicity, and wall-time.

1 Introduction

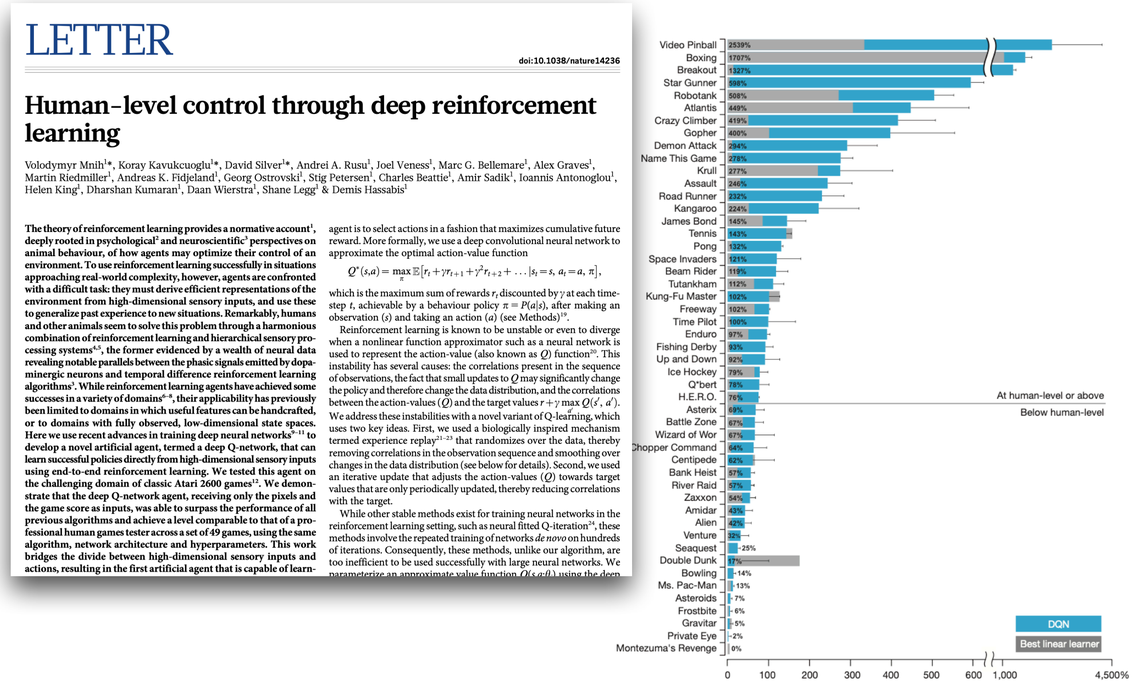
In recent years, several different approaches have been proposed for reinforcement learning with neural network function approximators. The leading contenders are deep Q-learning [Mni+15], "vanilla" policy gradient methods [Mni+16], and trust region / natural policy gradient methods [Sch+15b]. However, there is room for improvement in developing a method that is scalable (to large models and parallel implementations), data efficient, and robust (i.e., successful on a variety of problems without hyperparameter tuning). Q-learning (with function approximation) fails on many simple problems[1] and is poorly understood, vanilla policy gradient methods have poor data effiency and robustness; and trust region policy optimization (TRPO) is relatively complicated, and is not compatible with architectures that include noise (such as dropout) or parameter sharing (between the policy and value function, or with auxiliary tasks).

This paper seeks to improve the current state of affairs by introducing an algorithm that attains

arXiv:1707.06347v2 [cs.LG] 28 Aug 2017

- On-policy algorithms.
- Simple (relatively speaking).
- Stable updates.
- Works well empirically, easy to parallelize.
- Works with both discrete and continuous action spaces.

Deep Q Networks



LETTER

doi:10.1038/nature14236

Human-level control through deep reinforcement learning

Volodymyr Mnih[1]*, Koray Kavukcuoglu[1]*, David Silver[1]*, Andrei A. Rusu[1], Joel Veness[1], Marc G. Bellemare[1], Alex Graves[1], Martin Riedmiller[1], Andreas K. Fidjeland[1], Georg Ostrovski[1], Stig Petersen[1], Charles Beattie[1], Amir Sadik[1], Ioannis Antonoglou[1], Helen King[1], Dharshan Kumaran[1], Daan Wierstra[1], Shane Legg[1] & Demis Hassabis[1]

The theory of reinforcement learning provides a normative account[1], deeply rooted in psychological[2] and neuroscientific[3] perspectives on animal behaviour, of how agents may optimize their control of an environment. To use reinforcement learning successfully in situations approaching real-world complexity, however, agents are confronted with a difficult task: they must derive efficient representations of the environment from high-dimensional sensory inputs, and use these to generalize past experience to new situations. Remarkably, humans and other animals seem to solve this problem through a harmonious combination of reinforcement learning and hierarchical sensory processing systems[4,5], the former evidenced by a wealth of neural data revealing notable parallels between the phasic signals emitted by dopaminergic neurons and temporal difference reinforcement learning algorithms[3]. While reinforcement learning agents have achieved some successes in a variety of domains[6–8], their applicability has previously been limited to domains in which useful features can be handcrafted, or to domains with fully observed, low-dimensional state spaces. Here we use recent advances in training deep neural networks[9–11] to develop a novel artificial agent, termed a deep Q-network, that can learn successful policies directly from high-dimensional sensory inputs using end-to-end reinforcement learning. We tested this agent on the challenging domain of classic Atari 2600 games[12]. We demonstrate that the deep Q-network agent, receiving only the pixels and the game score as inputs, was able to surpass the performance of all previous algorithms and achieve a level comparable to that of a professional human games tester across a set of 49 games, using the same algorithm, network architecture and hyperparameters. This work bridges the divide between high-dimensional sensory inputs and actions, resulting in the first artificial agent that is capable of learn-

agent is to select actions in a fashion that maximizes cumulative future reward. More formally, we use a deep convolutional neural network to approximate the optimal action-value function

$$Q^*(s,a) = \max_{\pi} \mathbb{E}\left[r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \ldots \mid s_t = s,\, a_t = a,\, \pi\right],$$

which is the maximum sum of rewards r_t discounted by γ at each time-step t, achievable by a behaviour policy $\pi = P(a|s)$, after making an observation (s) and taking an action (a) (see Methods)[19].

Reinforcement learning is known to be unstable or even to diverge when a nonlinear function approximator such as a neural network is used to represent the action-value (also known as Q) function[20]. This instability has several causes: the correlations present in the sequence of observations, the fact that small updates to Q may significantly change the policy and therefore change the data distribution, and the correlations between the action-values (Q) and the target values $r + \gamma \max_{a'} Q(s', a')$. We address these instabilities with a novel variant of Q-learning, which uses two key ideas. First, we used a biologically inspired mechanism termed experience replay[21–23] that randomizes over the data, thereby removing correlations in the observation sequence and smoothing over changes in the data distribution (see below for details). Second, we used an iterative update that adjusts the action-values (Q) towards target values that are only periodically updated, thereby reducing correlations with the target.

While other stable methods exist for training neural networks in the reinforcement learning setting, such as neural fitted Q-iteration[24], these methods involve the repeated training of networks *de novo* on hundreds of iterations. Consequently, these methods, unlike our algorithm, are too inefficient to be used successfully with large neural networks. We parameterize an approximate value function $Q(s,a;\theta_i)$ using the deep

- Off-policy algorithm.
- Sample efficient (due to experience replay).
- Works with discrete action spaces only.

Why does DQN only work with discrete action spaces?

Recall DQN update: $\theta_{t+1} = \theta_t + \alpha \left[r + \gamma \max_{a' \in A} \bar{Q}_\theta(s', a') - Q(s, a) \right] \nabla Q(s, a)$

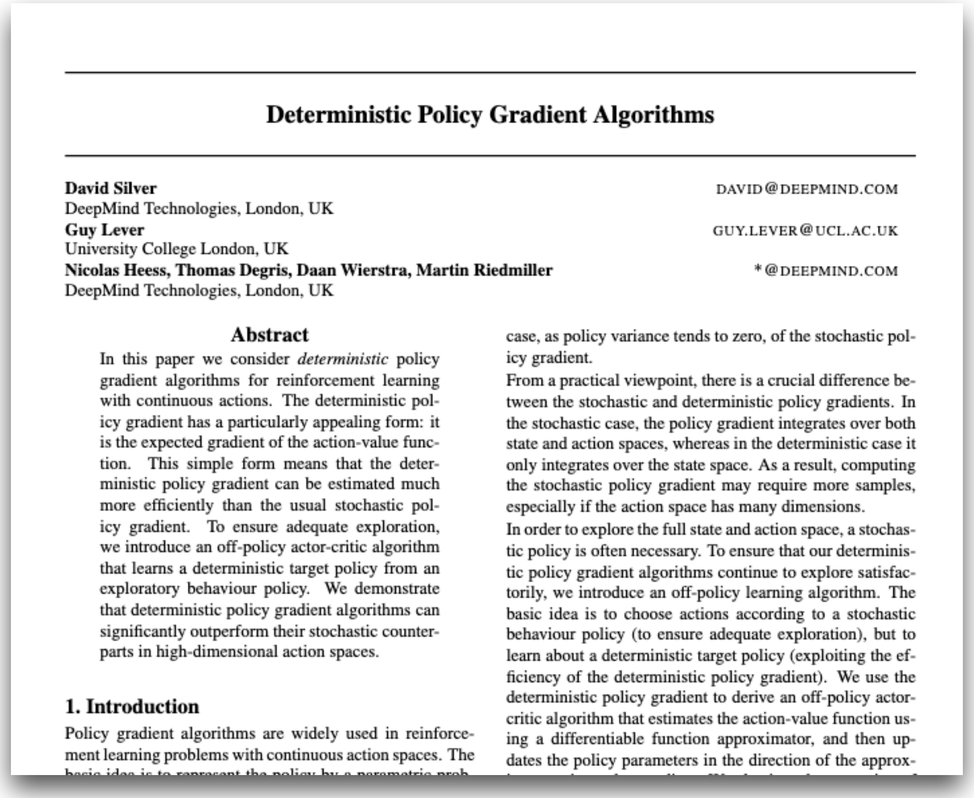
- why doesn't this work for continuous actions?
  - $\max_{a' \in A} Q(s', a')$
    - Discrete → enumerate and take max value
    - Continuous → There's no good way of evaluating this quickly in general.

How do we bring efficiency of DQN with continuous action space of actor-critic methods?

$Q_w(s, a)$ $\pi_\theta(a|s)$

parametrized

Deterministic policy gradient algorithms (DPG)



Deterministic Policy Gradient Algorithms

David Silver DAVID@DEEPMIND.COM
DeepMind Technologies, London, UK
Guy Lever GUY.LEVER@UCL.AC.UK
University College London, UK
Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller *@DEEPMIND.COM
DeepMind Technologies, London, UK

Abstract

In this paper we consider *deterministic* policy gradient algorithms for reinforcement learning with continuous actions. The deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function. This simple form means that the deterministic policy gradient can be estimated much more efficiently than the usual stochastic policy gradient. To ensure adequate exploration, we introduce an off-policy actor-critic algorithm that learns a deterministic target policy from an exploratory behaviour policy. We demonstrate that deterministic policy gradient algorithms can significantly outperform their stochastic counterparts in high-dimensional action spaces.

1. Introduction

Policy gradient algorithms are widely used in reinforcement learning problems with continuous action spaces. The

case, as policy variance tends to zero, of the stochastic policy gradient.

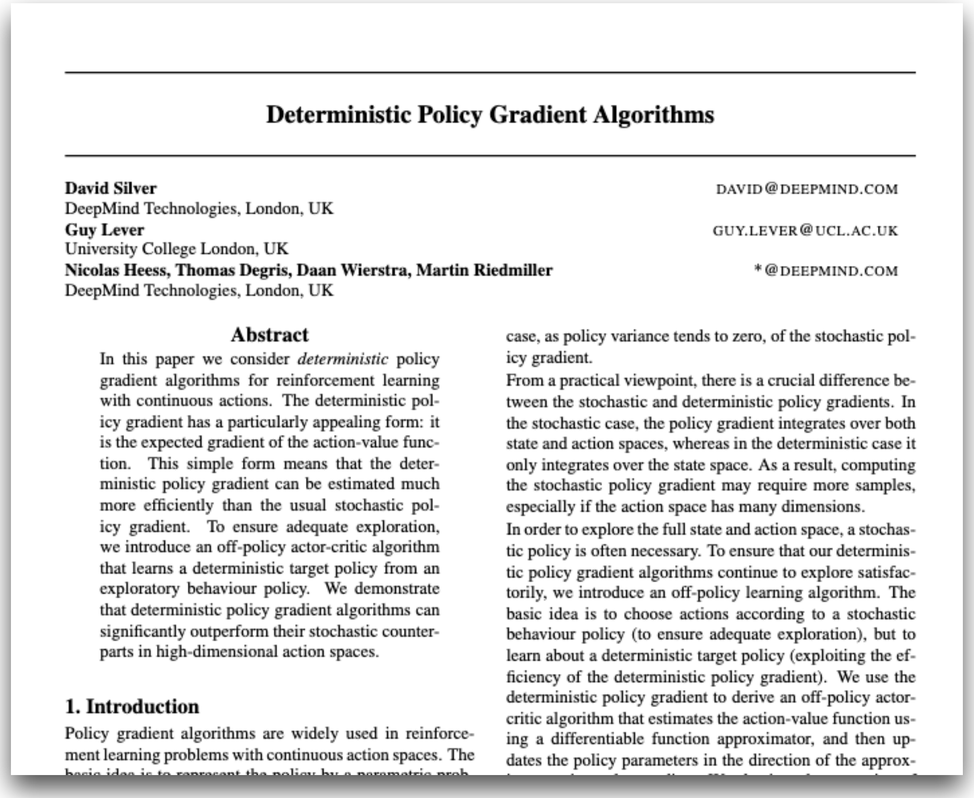
From a practical viewpoint, there is a crucial difference between the stochastic and deterministic policy gradients. In the stochastic case, the policy gradient integrates over both state and action spaces, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.

In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that our deterministic policy gradient algorithms continue to explore satisfactorily, we introduce an off-policy learning algorithm. The basic idea is to choose actions according to a stochastic behaviour policy (to ensure adequate exploration), but to learn about a deterministic target policy (exploiting the efficiency of the deterministic policy gradient). We use the deterministic policy gradient to derive an off-policy actor-critic algorithm that estimates the action-value function using a differentiable function approximator, and then updates the policy parameters in the direction of the approx-

Key ideas:

The algorithm uses a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, instead of a stochastic policy $a_t \sim \pi_\theta(\cdot \mid s_t)$.

Deterministic policy gradient algorithms (DPG)



Deterministic Policy Gradient Algorithms

David Silver DAVID@DEEPMIND.COM
DeepMind Technologies, London, UK
Guy Lever GUY.LEVER@UCL.AC.UK
University College London, UK
Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller *@DEEPMIND.COM
DeepMind Technologies, London, UK

Abstract

In this paper we consider *deterministic* policy gradient algorithms for reinforcement learning with continuous actions. The deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function. This simple form means that the deterministic policy gradient can be estimated much more efficiently than the usual stochastic policy gradient. To ensure adequate exploration, we introduce an off-policy actor-critic algorithm that learns a deterministic target policy from an exploratory behaviour policy. We demonstrate that deterministic policy gradient algorithms can significantly outperform their stochastic counterparts in high-dimensional action spaces.

1. Introduction

Policy gradient algorithms are widely used in reinforcement learning problems with continuous action spaces. The basic idea is to represent the policy by a parametric prob-

case, as policy variance tends to zero, of the stochastic policy gradient.

From a practical viewpoint, there is a crucial difference between the stochastic and deterministic policy gradients. In the stochastic case, the policy gradient integrates over both state and action spaces, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.

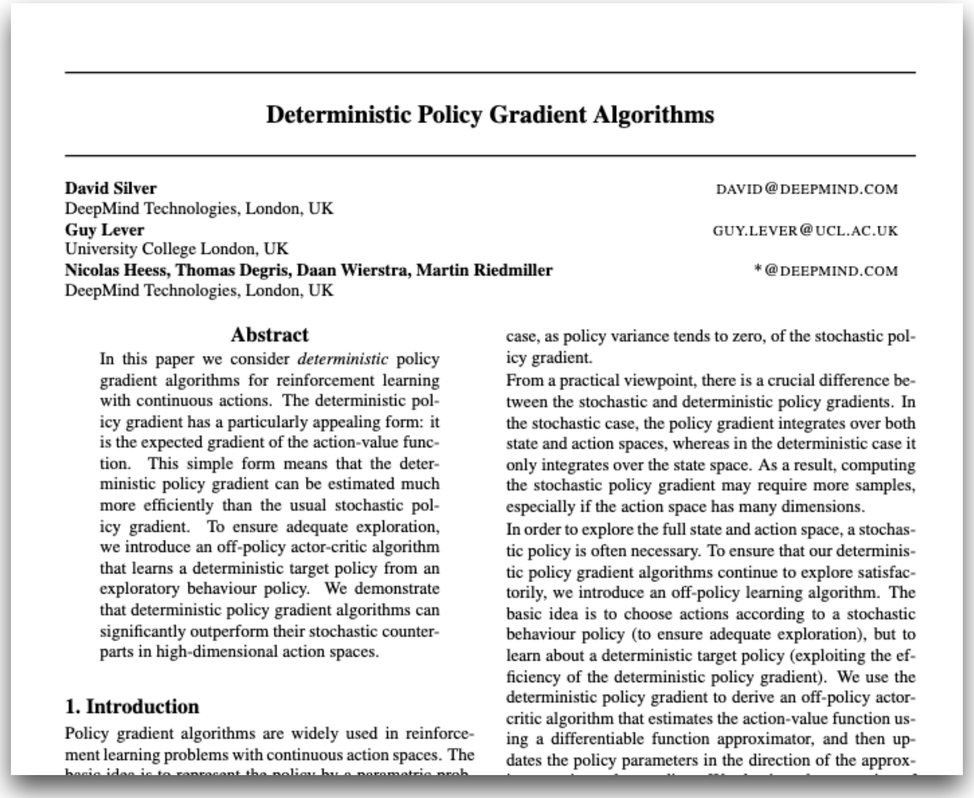
In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that our deterministic policy gradient algorithms continue to explore satisfactorily, we introduce an off-policy learning algorithm. The basic idea is to choose actions according to a stochastic behaviour policy (to ensure adequate exploration), but to learn about a deterministic target policy (exploiting the efficiency of the deterministic policy gradient). We use the deterministic policy gradient to derive an off-policy actor-critic algorithm that estimates the action-value function using a differentiable function approximator, and then updates the policy parameters in the direction of the approx-

Key ideas:

The algorithm uses a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, instead of a stochastic policy $a_t \sim \pi_\theta(\cdot \mid s_t)$.

Introduce *deterministic policy gradient theorem*.

Deterministic policy gradient algorithms (DPG)



Deterministic Policy Gradient Algorithms

David Silver DAVID@DEEPMIND.COM
DeepMind Technologies, London, UK
Guy Lever GUY.LEVER@UCL.AC.UK
University College London, UK
Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller *@DEEPMIND.COM
DeepMind Technologies, London, UK

Abstract

In this paper we consider *deterministic* policy gradient algorithms for reinforcement learning with continuous actions. The deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function. This simple form means that the deterministic policy gradient can be estimated much more efficiently than the usual stochastic policy gradient. To ensure adequate exploration, we introduce an off-policy actor-critic algorithm that learns a deterministic target policy from an exploratory behaviour policy. We demonstrate that deterministic policy gradient algorithms can significantly outperform their stochastic counterparts in high-dimensional action spaces.

1. Introduction

Policy gradient algorithms are widely used in reinforcement learning problems with continuous action spaces. The basic idea is to represent the policy by a parametric prob-

case, as policy variance tends to zero, of the stochastic policy gradient.

From a practical viewpoint, there is a crucial difference between the stochastic and deterministic policy gradients. In the stochastic case, the policy gradient integrates over both state and action spaces, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.

In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that our deterministic policy gradient algorithms continue to explore satisfactorily, we introduce an off-policy learning algorithm. The basic idea is to choose actions according to a stochastic behaviour policy (to ensure adequate exploration), but to learn about a deterministic target policy (exploiting the efficiency of the deterministic policy gradient). We use the deterministic policy gradient to derive an off-policy actor-critic algorithm that estimates the action-value function using a differentiable function approximator, and then updates the policy parameters in the direction of the approx-

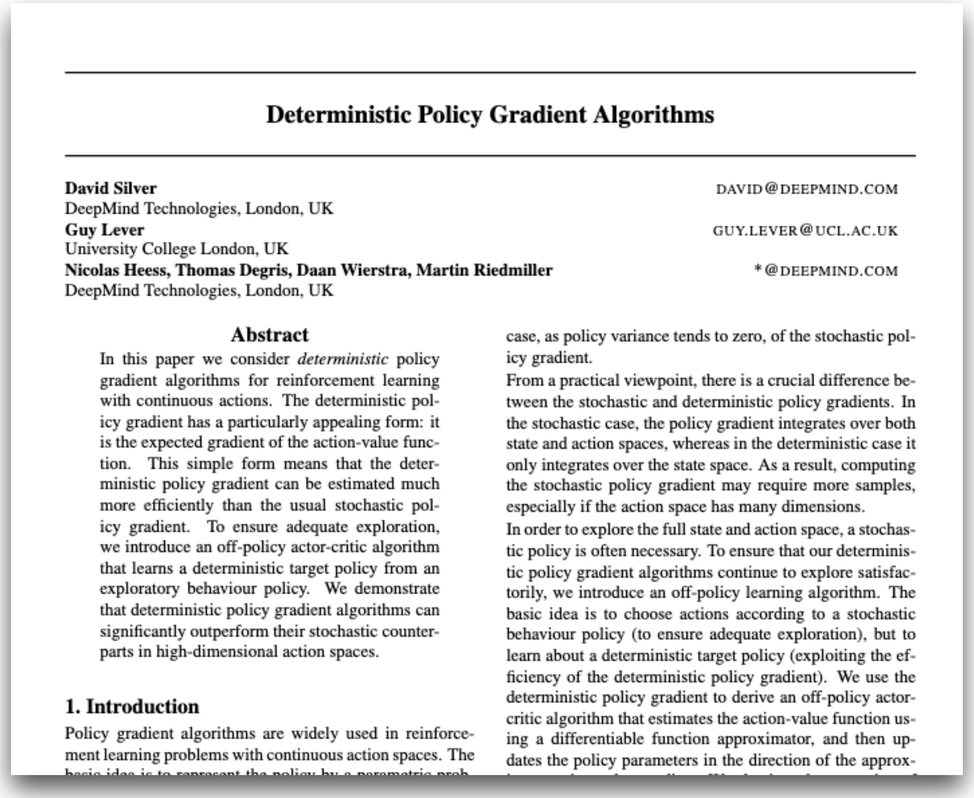
Key ideas:

The algorithm uses a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, instead of a stochastic policy $a_t \sim \pi_\theta(\cdot \mid s_t)$.

Introduce *deterministic policy gradient theorem*.

> Deterministic policy gradients may be approximated much more efficiently than their stochastic counterparts.

Deterministic policy gradient algorithms (DPG)



Deterministic Policy Gradient Algorithms

David Silver DAVID@DEEPMIND.COM
DeepMind Technologies, London, UK
Guy Lever GUY.LEVER@UCL.AC.UK
University College London, UK
Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller *@DEEPMIND.COM
DeepMind Technologies, London, UK

Abstract

In this paper we consider *deterministic* policy gradient algorithms for reinforcement learning with continuous actions. The deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function. This simple form means that the deterministic policy gradient can be estimated much more efficiently than the usual stochastic policy gradient. To ensure adequate exploration, we introduce an off-policy actor-critic algorithm that learns a deterministic target policy from an exploratory behaviour policy. We demonstrate that deterministic policy gradient algorithms can significantly outperform their stochastic counterparts in high-dimensional action spaces.

1. Introduction

Policy gradient algorithms are widely used in reinforcement learning problems with continuous action spaces. The basic idea is to represent the policy by a parametric prob-

case, as policy variance tends to zero, of the stochastic policy gradient.

From a practical viewpoint, there is a crucial difference between the stochastic and deterministic policy gradients. In the stochastic case, the policy gradient integrates over both state and action spaces, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.

In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that our deterministic policy gradient algorithms continue to explore satisfactorily, we introduce an off-policy learning algorithm. The basic idea is to choose actions according to a stochastic behaviour policy (to ensure adequate exploration), but to learn about a deterministic target policy (exploiting the efficiency of the deterministic policy gradient). We use the deterministic policy gradient to derive an off-policy actor-critic algorithm that estimates the action-value function using a differentiable function approximator, and then updates the policy parameters in the direction of the approx-

Key ideas:

The algorithm uses a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, instead of a stochastic policy $a_t \sim \pi_\theta(\cdot \mid s_t)$.

Introduce *deterministic policy gradient theorem*.

> Deterministic policy gradients may be approximated much more efficiently than their stochastic counterparts.

To ensure adequate exploration, introduce an **off-policy actor-critic algorithm**, that learns form an exploratory behavior policy.

Deterministic policy gradient algorithms (DPG)

Deterministic Policy Gradient Algorithms

David Silver DAVID@DEEPMIND.COM
DeepMind Technologies, London, UK
Guy Lever GUY.LEVER@UCL.AC.UK
University College London, UK
Nicolas Heess, Thomas Degris, Daan Wierstra, Martin Riedmiller *@DEEPMIND.COM
DeepMind Technologies, London, UK

Abstract

In this paper we consider *deterministic* policy gradient algorithms for reinforcement learning with continuous actions. The deterministic policy gradient has a particularly appealing form: it is the expected gradient of the action-value function. This simple form means that the deterministic policy gradient can be estimated much more efficiently than the usual stochastic policy gradient. To ensure adequate exploration, we introduce an off-policy actor-critic algorithm that learns a deterministic target policy from an exploratory behaviour policy. We demonstrate that deterministic policy gradient algorithms can significantly outperform their stochastic counterparts in high-dimensional action spaces.

1. Introduction

Policy gradient algorithms are widely used in reinforcement learning problems with continuous action spaces. The

case, as policy variance tends to zero, of the stochastic policy gradient.

From a practical viewpoint, there is a crucial difference between the stochastic and deterministic policy gradients. In the stochastic case, the policy gradient integrates over both state and action spaces, whereas in the deterministic case it only integrates over the state space. As a result, computing the stochastic policy gradient may require more samples, especially if the action space has many dimensions.

In order to explore the full state and action space, a stochastic policy is often necessary. To ensure that our deterministic policy gradient algorithms continue to explore satisfactorily, we introduce an off-policy learning algorithm. The basic idea is to choose actions according to a stochastic behaviour policy (to ensure adequate exploration), but to learn about a deterministic target policy (exploiting the efficiency of the deterministic policy gradient). We use the deterministic policy gradient to derive an off-policy actor-critic algorithm that estimates the action-value function using a differentiable function approximator, and then updates the policy parameters in the direction of the approx-

Key ideas:

The algorithm uses a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, instead of a stochastic policy $a_t \sim \pi_\theta(\cdot \mid s_t)$.

Introduce *deterministic policy gradient theorem*.

> Deterministic policy gradients may be approximated much more efficiently than their stochastic counterparts.

To ensure adequate exploration, introduce an **off-policy actor-critic algorithm**, that learns form an exploratory behavior policy.

Intuition: If we cannot efficiently calculate $\max_{a \in A} Q_\theta(s, a)$, perhaps we can learn a policy that moves in the direction that increases the Q-value.

Deterministic policy gradient algorithms (DPG)

If we had a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, we could easily define the corresponding Bellman target:

$$y_i = r_i + \gamma Q_{targ}(s_i, \mu_\theta(s_i))$$

Deterministic policy gradient algorithms (DPG)

If we had a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, we could easily define the corresponding Bellman target:

$$y_i = r_i + \gamma Q_{targ}(s_i, \mu_\theta(s_i))$$

This has the familiar Actor-Critic architecture we saw a few lectures ago:

1. Actor: $\mu_\theta(s)$ is our policy network.
2. Critic: $Q_w(s, a)$ is our Q-function parameterization that evaluates the actor's actions.

Deterministic policy gradient algorithms (DPG)

If we had a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, we could easily define the corresponding Bellman target:

$$y_i = r_i + \gamma Q_{targ}(s_i, \mu_\theta(s_i))$$

This has the familiar Actor-Critic architecture we saw a few lectures ago:

1. Actor: $\mu_\theta(s)$ is our policy network.
2. Critic: $Q_w(s, a)$ is our Q-function parameterization that evaluates the actor's actions.

The critic can be learned by minimizing TD error. But what about the actor μ?

Intuition: the actor's goal is to choose actions that make the critic's Q-values as high as possible.

I.e., find parameters θ that maximize $J(\mu_\theta) = \mathbb{E}\left[Q(s, \mu_\theta(s))\right]$.

Deterministic policy gradient algorithms (DPG)

If we had a ***deterministic policy*** parameterization $a_t = \mu_\theta(s_t)$, we could easily define the corresponding Bellman target:

$$y_i = r_i + \gamma Q_{targ}(s_i, \mu_\theta(s_i))$$

This has the familiar Actor-Critic architecture we saw a few lectures ago:

1. Actor: $\mu_\theta(s)$ is our policy network.
2. Critic: $Q_w(s, a)$ is our Q-function parameterization that evaluates the actor's actions.

The critic can be learned by minimizing TD error. But what about the actor μ?

Intuition: the actor's goal is to choose actions that make the critic's Q-values as high as possible.

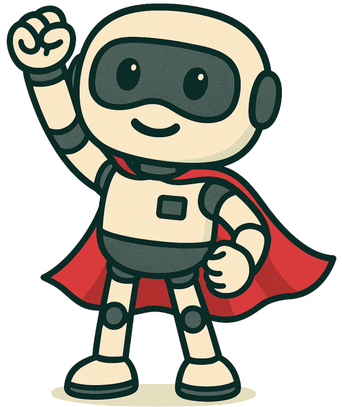
I.e., find parameters θ that maximize $J(\mu_\theta) = \mathbb{E}\left[Q(s, \mu_\theta(s))\right]$.

How can we compute a gradient for this?

Recall, the (stochastic) policy gradient theorem

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \mathbb{E}_{s\sim\rho^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)} \left[Q_{\pi_\theta}(s,a) \nabla log(\pi_\theta(s,a)) \right]$$



Normalized expected state visitation frequency over an episode induced by policy π_θ.

True state-action value function of policy π_θ.

$$\rho^{\pi_\theta}(s') = \int_S \sum_{t=0}^{\infty} \gamma^t \mathbb{P}(S_0 = s) \mathbb{P}(s \to s', t, \pi_\theta) ds$$

Crucially: $\nabla J(\theta)$ doesn't depend on the gradient through the environment dynamics!

Algorithmic implication: We can estimate $\nabla J(\theta) = \mathbb{E}_{S\sim\mu, A\sim\pi_\theta(\cdot|S)} \left[Q_{\pi_\theta}(S,A) \nabla log(\pi_\theta(A \mid S)) \right]$ by running π_θ in the environment, estimating $Q_{\pi_\theta}(s,a)$, and computing averages of samples of $Q^{\pi_\theta}(s^{(i)}, a^{(i)}) \nabla \log(\pi_\theta(s^{(i)}, a^{(i)}))$.

Recall, the (stochastic) policy gradient theorem

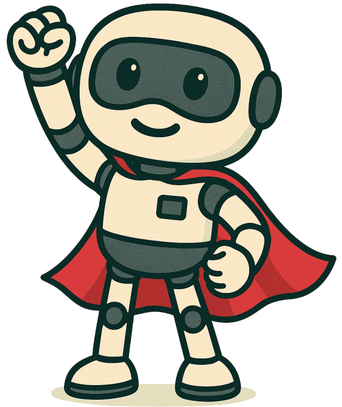
$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \mathbb{E}_{s\sim\rho^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)} \left[Q_{\pi_\theta}(s,a) \nabla log(\pi_\theta(s,a)) \right]$$

Replace MC estimates with function approximation.

Actor critic methods:

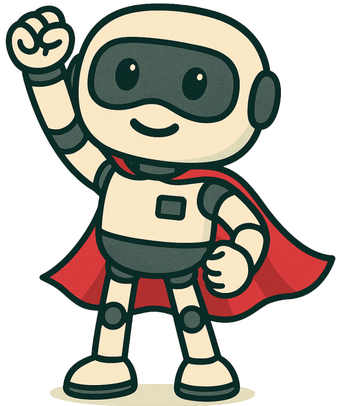
$$\nabla J(\theta) \approx \mathbb{E}_{s\sim\rho^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)} \left[Q_w(s,a) \nabla_\theta \log \pi_\theta(a \mid s) \right]$$



Recall, the (stochastic) policy gradient theorem

$$\nabla J(\theta) = \nabla V^{\pi}(s_0)$$

$$\propto \mathbb{E}_{s\sim\rho^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)} \left[Q_{\pi_\theta}(s,a) \nabla log(\pi_\theta(s,a)) \right]$$



Replace MC estimates with function approximation.

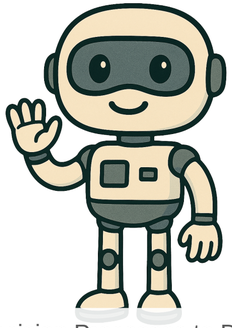
Actor critic methods:

$$\nabla J(\theta) \approx \mathbb{E}_{s\sim\rho^{\pi_\theta}, a\sim\pi_\theta(\cdot|s)} \left[Q_w(s,a) \nabla_\theta \log \pi_\theta(a \mid s) \right]$$

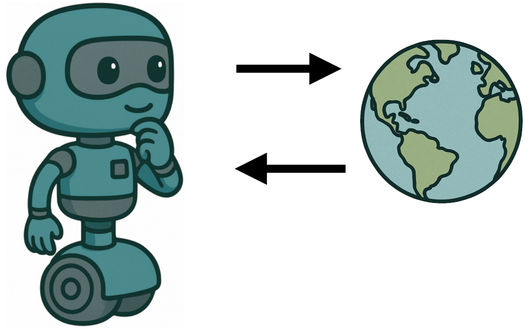
Off-policy actor critic:

$$\nabla J(\theta) \approx \mathbb{E}_{s\sim\rho^{\beta_\theta}, a\sim\beta_\theta(\cdot|s)} \left[\frac{\pi_\theta(a \mid s)}{\beta_\theta(a \mid s)} Q_w(s,a) \nabla_\theta \log \pi_\theta(a \mid s) \right]$$

Learn a target policy $\pi(a \mid s)$



Using episodes sampled under a different behavioral policy $\beta_\theta(a \mid s)$



The deterministic policy gradient theorem

Given the objective function $J(\mu_\theta) = \int_S \rho^{\mu_\theta}(s) r(s, \mu_\theta(s)) ds = \mathbb{E}_{s\sim\rho^\mu}\left[r(s, \mu_\theta(s))\right]$

Theorem 1 (Deterministic Policy Gradient Theorem). *Suppose that the MDP satisfies conditions A.1 (see Appendix; these imply that $\nabla_\theta \mu_\theta(s)$ and $\nabla_a Q^\mu(s,a)$ exist and that the deterministic policy gradient exists. Then,*

$$\begin{aligned}\nabla_\theta J(\mu_\theta) &= \int_{\mathcal{S}} \rho^\mu(s) \nabla_\theta \mu_\theta(s)\ \nabla_a Q^\mu(s,a)|_{a=\mu_\theta(s)}\, \mathrm{d}s \\ &= \mathbb{E}_{s\sim\rho^\mu}\left[\nabla_\theta \mu_\theta(s)\ \nabla_a Q^\mu(s,a)|_{a=\mu_\theta(s)}\right] \quad (9)\end{aligned}$$

Policy gradient is in terms of $\nabla_\theta \mu_\theta(s)$ and $\nabla_a Q^{\pi_\theta}(s,a)$.

The deterministic policy gradient theorem

Given the objective function $J(\mu_\theta) = \int_S \rho^{\mu_\theta}(s) r(s, \mu_\theta(s)) ds = \mathbb{E}_{s\sim\rho^\mu}\left[r(s, \mu_\theta(s))\right]$

Theorem 1 (Deterministic Policy Gradient Theorem). *Suppose that the MDP satisfies conditions A.1 (see Appendix; these imply that $\nabla_\theta \mu_\theta(s)$ and $\nabla_a Q^\mu(s,a)$ exist and that the deterministic policy gradient exists. Then,*

$$\begin{aligned}\nabla_\theta J(\mu_\theta) &= \int_{\mathcal{S}} \rho^\mu(s) \nabla_\theta \mu_\theta(s)\ \nabla_a Q^\mu(s,a)|_{a=\mu_\theta(s)}\, \mathrm{d}s \\ &= \mathbb{E}_{s\sim\rho^\mu}\left[\nabla_\theta \mu_\theta(s)\ \nabla_a Q^\mu(s,a)|_{a=\mu_\theta(s)}\right] \quad (9)\end{aligned}$$

Policy gradient is in terms of $\nabla_\theta \mu_\theta(s)$ and $\nabla_a Q^{\pi_\theta}(s,a)$.

The deterministic policy gradient theorem is the limiting case of the stochastic policy gradient theorem (as policy noise is brought to zero).

Theorem 2. *Consider a stochastic policy $\pi_{\mu_\theta,\sigma}$ such that $\pi_{\mu_\theta,\sigma}(a|s) = \nu_\sigma(\mu_\theta(s), a)$, where σ is a parameter controlling the variance and ν_σ satisfy conditions B.1 and the MDP satisfies conditions A.1 and A.2. Then,*

$$\lim_{\sigma\downarrow 0} \nabla_\theta J(\pi_{\mu_\theta,\sigma}) = \nabla_\theta J(\mu_\theta) \quad (10)$$

where on the l.h.s. the gradient is the standard stochastic policy gradient and on the r.h.s. the gradient is the deterministic policy gradient.

What's the intuition behind the DPG theorem?

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^{\mu_\theta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_w(s,a)\right]$$

Want to take policy improvement steps to nudge $\mu_\theta(s)$ select actions with higher Q-values.

$$\theta_{k+1} = \theta_k + \alpha \mathbb{E}_{s\sim\rho}\left[\nabla_\theta Q^{\mu_\theta}(s, \mu_\theta(s))\right]$$

Apply chain rule

$$\frac{dQ}{d\theta} = \frac{dQ}{da} \cdot \frac{da}{d\theta}$$

$\nabla_a Q^{\mu_\theta}(s,a)$ $\quad$ $\nabla_\theta \mu_\theta(s)$

$$\theta_{k+1} = \theta_k + \alpha \mathbb{E}\left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a)\big|_{a=\mu_\theta(s)}\right]$$

Note: formal proof in appendix.

Intuition:

$\nabla_a Q(s,a)$ – provides a gradient vector. It tells the actor: from state s, this is the direction in the action space that you should move in to get a higher value.

$\nabla_\theta \mu_\theta(s)$ – Jacobian of the policy function (map between spaces θ and d). applied to $\nabla_a Q(s,a)$ it tells us how to change θ to produce actions with higher val.

DPG actor-critic algorithms

On-policy case

Key idea: Parameterize a deterministic actor $\mu_\theta(s)$ and a critic $Q_w(s,a)$. And use the DPG theorem to update the policy parameters θ with on-policy samples.

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^{\mu_\theta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_w(s,a)\right]$$

DPG actor-critic algorithms

On-policy case

Key idea: Parameterize a deterministic actor $\mu_\theta(s)$ and a critic $Q_w(s,a)$. And use the DPG theorem to update the policy parameters θ with on-policy samples.

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^{\mu_\theta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_w(s,a)\right]$$

Update equations:

SARSA-style value update

$$\delta_t = r_t + \gamma Q^w(s_{t+1}, a_{t+1}) - Q^w(s_t, a_t)$$

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w Q^w(s_t, a_t)$$ ← Critic update equation

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu_\theta(s_t)\ \nabla_a Q^w(s_t, a_t)|_{a=\mu_\theta(s)}$$

Actor update equation

DPG actor-critic algorithms

On-policy case

Key idea: Parameterize a deterministic actor $\mu_\theta(s)$ and a critic $Q_w(s,a)$. And use the DPG theorem to update the policy parameters θ with on-policy samples.

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^{\mu_\theta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_w(s,a)\right]$$

Update equations:

SARSA-style value update

$$\delta_t = r_t + \gamma Q^w(s_{t+1}, a_{t+1}) - Q^w(s_t, a_t)$$

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w Q^w(s_t, a_t)$$

Critic update equation

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu_\theta(s_t)\ \nabla_a Q^w(s_t, a_t)|_{a=\mu_\theta(s)}$$

Actor update equation

Off-policy case

Key idea: Parameterize a deterministic actor $\mu_\theta(s)$ and a critic $Q_w(s,a)$. And use the DPG theorem to update the policy parameters θ.

$$\nabla_\theta J_\beta(\mu_\theta) \approx \mathbb{E}_{s\sim\rho^{\beta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_w(s,a)\right]$$

Update equations:

Q-learning-style update

$$\delta_t = r_t + \gamma Q^w(s_{t+1}, \mu_\theta(s_{t+1})) - Q^w(s_t, a_t)$$

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w Q^w(s_t, a_t)$$

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu_\theta(s_t)\ \nabla_a Q^w(s_t, a_t)|_{a=\mu_\theta(s)}$$

DPG actor-critic algorithms

On-policy case

Key idea: Parameterize a deterministic actor $\mu_\theta(s)$ and a critic $Q_w(s,a)$. And use the DPG theorem to update the policy parameters θ with on-policy samples.

$$\nabla_\theta J(\mu_\theta) = \mathbb{E}_{s\sim\rho^{\mu_\theta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_w(s,a)\right]$$

Update equations:

SARSA-style value update

$$\delta_t = r_t + \gamma Q^w(s_{t+1}, a_{t+1}) - Q^w(s_t, a_t)$$

Critic update equation

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w Q^w(s_t, a_t)$$

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu_\theta(s_t)\ \nabla_a Q^w(s_t, a_t)|_{a=\mu_\theta(s)}$$

Actor update equation

Off-policy case

Key idea: Parameterize a deterministic actor $\mu_\theta(s)$ and a critic $Q_w(s,a)$. And use the DPG theorem to update the policy parameters θ.

$$\nabla_\theta J_\beta(\mu_\theta) \approx \mathbb{E}_{s\sim\rho^{\beta}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q_w(s,a)\right]$$

Update equations:

Q-learning-style update

$$\delta_t = r_t + \gamma Q^w(s_{t+1}, \mu_\theta(s_{t+1})) - Q^w(s_t, a_t)$$

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w Q^w(s_t, a_t)$$

$$\theta_{t+1} = \theta_t + \alpha_\theta \nabla_\theta \mu_\theta(s_t)\ \nabla_a Q^w(s_t, a_t)|_{a=\mu_\theta(s)}$$

The authors propose to use linear function approximators.

DPG experimental results

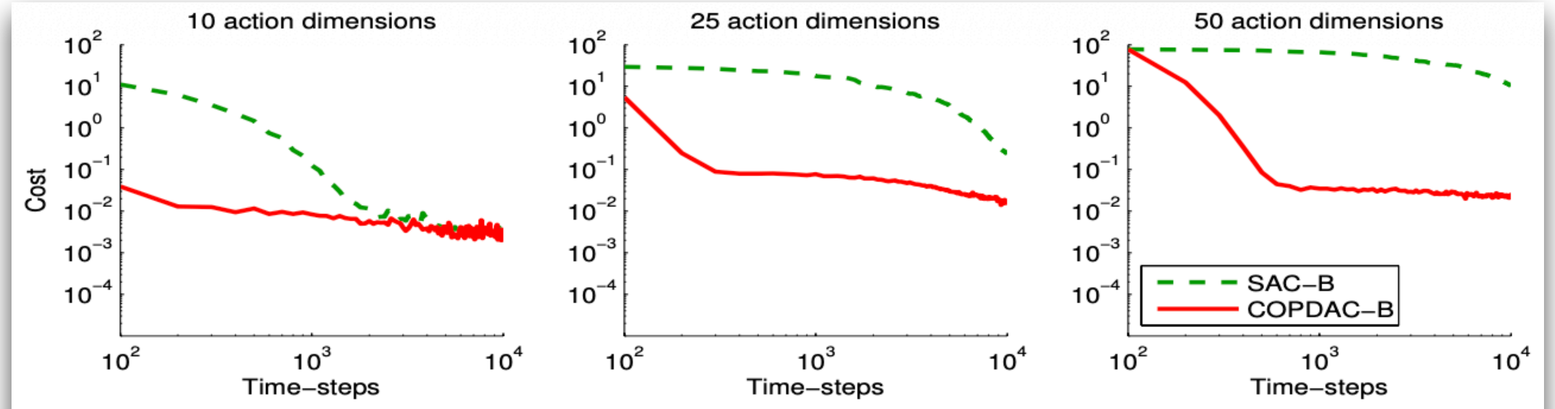
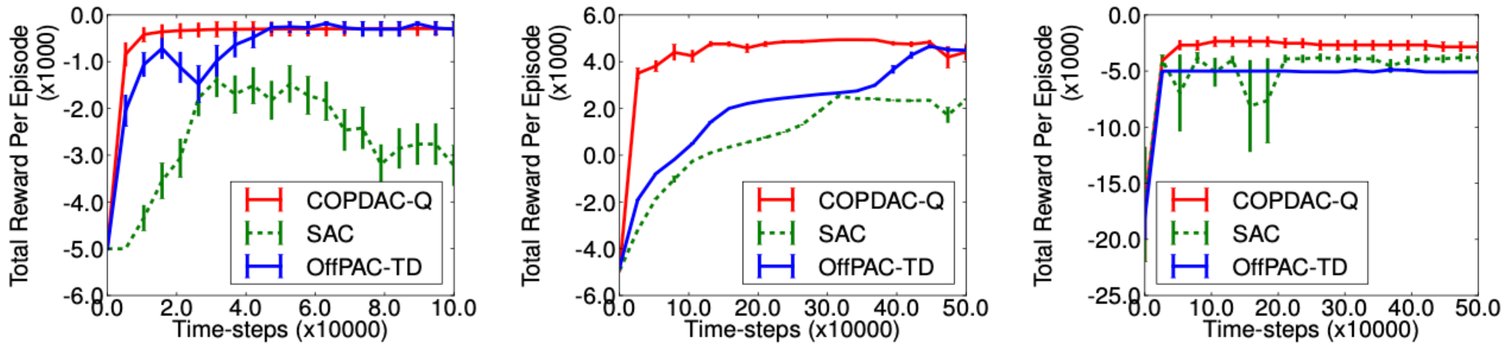


Figure 1. Comparison of stochastic actor-critic (SAC-B) and deterministic actor-critic (COPDAC-B) on the continuous bandit task.



(a) Mountain Car (b) Pendulum (c) 2D Puddle World

Figure 2. Comparison of stochastic on-policy actor-critic (SAC), stochastic off-policy actor-critic (OffPAC), and deterministic off-policy actor-critic (COPDAC) on continuous-action reinforcement learning. Each point is the average test performance of the mean policy.

Stochastic policy gradient

Deterministic policy gradient

Key message: Actor-critic algorithms with deterministic policy gradients outperform algorithms with stochastic policy gradients in the tested continuous control tasks.

Why deterministic policy gradients over stochastic policy gradients?

- Simple answer: deterministic PG $\Rightarrow$ improved sample efficiency.
  - $\hookrightarrow$ SPGs are generally more data hungry, to get good estimates of policy gradient.

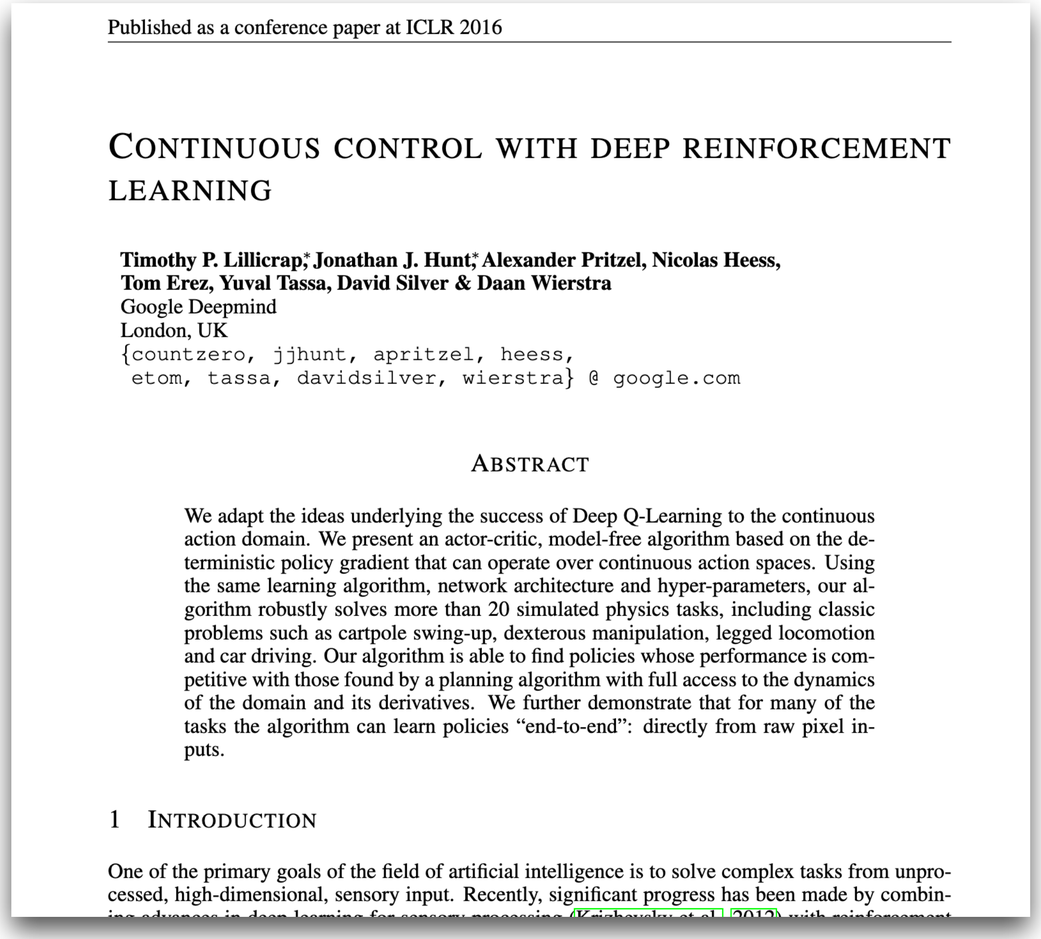
why? SGP require estimating $\mathbb{E}$ over both S and A:

$$\mathbb{E}_{s\sim\rho^{\pi},\, a\sim\pi(\cdot|s)}\left[\nabla_\theta \pi_\theta(a|s)\, Q^{\pi}(s,a)\right]$$

DPG requires expectation only over S: $\mathbb{E}_{s\sim\rho^{\pi}}\left[\nabla_\theta \mu_\theta(s)\, \nabla_a Q^{\pi}(s,a)\right]$

Tradeoff:
- Deterministic policies are less robust to distribution shifts, errors in $Q(s,a)$, etc.
- Explore. Stochastic policies naturally explore. Deterministic policies do not.
  - $\hookrightarrow$ This is why DPG-based are all off-policy.

Deep deterministic policy gradient algorithms (DDPG)



Published as a conference paper at ICLR 2016

CONTINUOUS CONTROL WITH DEEP REINFORCEMENT LEARNING

Timothy P. Lillicrap*, Jonathan J. Hunt*, Alexander Pritzel, Nicolas Heess,
Tom Erez, Yuval Tassa, David Silver & Daan Wierstra
Google Deepmind
London, UK
{countzero, jjhunt, apritzel, heess,
etom, tassa, davidsilver, wierstra} @ google.com

ABSTRACT

We adapt the ideas underlying the success of Deep Q-Learning to the continuous action domain. We present an actor-critic, model-free algorithm based on the deterministic policy gradient that can operate over continuous action spaces. Using the same learning algorithm, network architecture and hyper-parameters, our algorithm robustly solves more than 20 simulated physics tasks, including classic problems such as cartpole swing-up, dexterous manipulation, legged locomotion and car driving. Our algorithm is able to find policies whose performance is competitive with those found by a planning algorithm with full access to the dynamics of the domain and its derivatives. We further demonstrate that for many of the tasks the algorithm can learn policies "end-to-end": directly from raw pixel inputs.

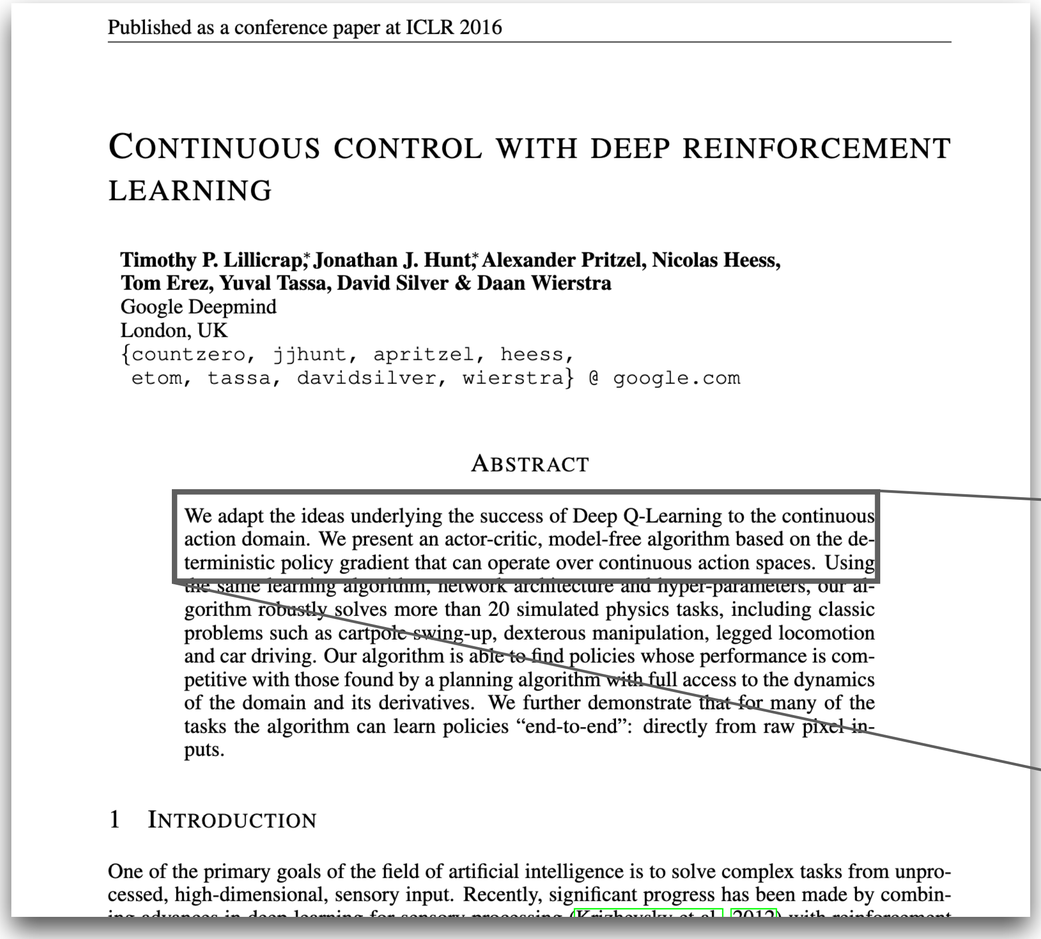
1 INTRODUCTION

One of the primary goals of the field of artificial intelligence is to solve complex tasks from unprocessed, high-dimensional, sensory input. Recently, significant progress has been made by combining advances in deep learning for sensory processing (Krizhevsky et al., 2012) with reinforcement

Intuitively: DQN for continuous control.

Take the DPG off-policy actor-critic framework, and combine it with key techniques from the DQN paper: experience replay and target networks.

Deep deterministic policy gradient algorithms (DDPG)



Published as a conference paper at ICLR 2016

CONTINUOUS CONTROL WITH DEEP REINFORCEMENT LEARNING

Timothy P. Lillicrap*, Jonathan J. Hunt*, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver & Daan Wierstra
Google Deepmind
London, UK
{countzero, jjhunt, apritzel, heess, etom, tassa, davidsilver, wierstra} @ google.com

ABSTRACT

We adapt the ideas underlying the success of Deep Q-Learning to the continuous action domain. We present an actor-critic, model-free algorithm based on the deterministic policy gradient that can operate over continuous action spaces. Using the same learning algorithm, network architecture and hyper-parameters, our algorithm robustly solves more than 20 simulated physics tasks, including classic problems such as cartpole swing-up, dexterous manipulation, legged locomotion and car driving. Our algorithm is able to find policies whose performance is competitive with those found by a planning algorithm with full access to the dynamics of the domain and its derivatives. We further demonstrate that for many of the tasks the algorithm can learn policies "end-to-end": directly from raw pixel inputs.

1 INTRODUCTION

One of the primary goals of the field of artificial intelligence is to solve complex tasks from unprocessed, high-dimensional, sensory input. Recently, significant progress has been made by combining advances in deep learning for sensory processing (Krizhevsky et al., 2012) with reinforcement

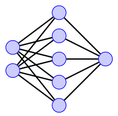
Intuitively: DQN for continuous control.

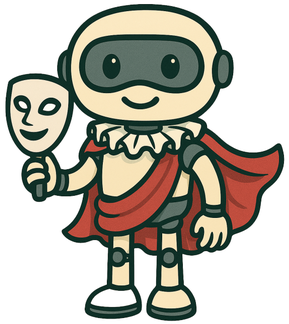
Take the DPG off-policy actor-critic framework, and combine it with key techniques from the DQN paper: experience replay and target networks.

"We adapt the ideas underlying the success of Deep Q-Learning to the continuous action domain. We present an actor-critic, model-free algorithm based on the deterministic policy gradient that can operate over continuous action spaces."

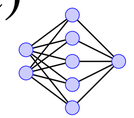
Deep deterministic policy gradient algorithms (DDPG)

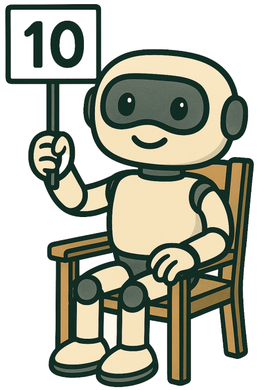
Deterministic actor network $a = \mu_\theta(s)$





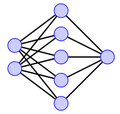
Parameterized critic network $Q_w(s, a)$

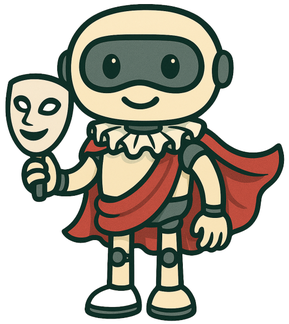




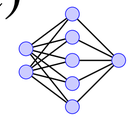
Deep deterministic policy gradient algorithms (DDPG)

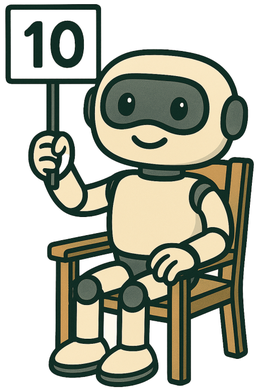
Deterministic actor network $a = \mu_\theta(s)$





Parameterized critic network $Q_w(s, a)$

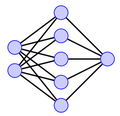


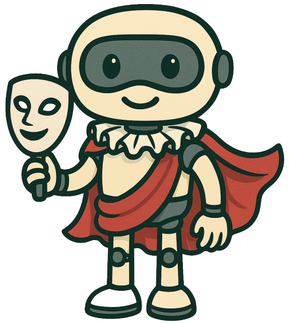


How will we train them both?

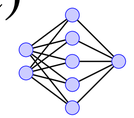
Deep deterministic policy gradient algorithms (DDPG)

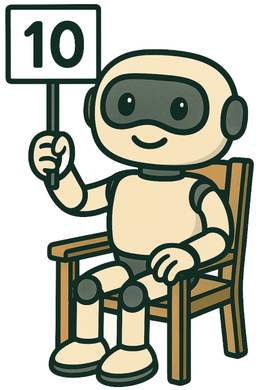
Deterministic <u>actor</u> network $a = \mu_\theta(s)$





Parameterized <u>critic</u> network $Q_w(s, a)$





<u>How will we train them both?</u>

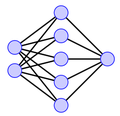
Actor gradient step (DPG theorem):

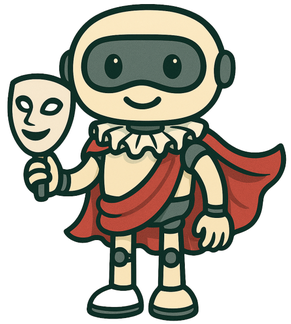
$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t \sim \beta}\left[\nabla_\theta Q_w(s, a) \mid_{s=s_t, a=\mu_\theta(s_t)} \right]$$

$$= \mathbb{E}_{s_t \sim \rho^\beta}\left[\nabla_a Q_w(s, a) \mid_{s=s_t, a=\mu_\theta(s_t)} \nabla_\theta \mu_\theta(s) \mid_{s=s_t} \right]$$

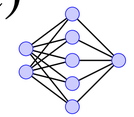
Deep deterministic policy gradient algorithms (DDPG)

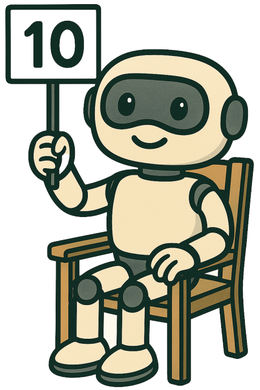
Deterministic actor network $a = \mu_\theta(s)$





Parameterized critic network $Q_w(s, a)$





How will we train them both?

Actor gradient step (DPG theorem):

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t \sim \beta}\left[\nabla_\theta Q_w(s, a) \mid_{s=s_t, a=\mu_\theta(s_t)} \right]$$

$$= \mathbb{E}_{s_t \sim \rho^\beta}\left[\nabla_a Q_w(s, a) \mid_{s=s_t, a=\mu_\theta(s_t)} \nabla_\theta \mu_\theta(s) \mid_{s=s_t} \right]$$

Critic loss (TD loss):

$$L(w) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta(\cdot|s_t)}\left[\left(Q_w(s_t, a_t) - y_t \right)^2 \right]$$

$$y_t = r(s_t, a_t) + \gamma Q_w(s_{t+1}, \mu(s_{t_1}))$$

Experience replay in DDPG

Why? Alleviate problems of correlated data and non-stationary distributions.
Leverage the strengths of off-policy learning: We can learn from past data collected by different, behaviour policies.

One challenge when using neural networks for reinforcement learning is that most optimization algorithms assume that the samples are independently and identically distributed. Obviously, when the samples are generated from exploring sequentially in an environment this assumption no longer holds. Additionally, to make efficient use of hardware optimizations, it is essential to learn in mini-batches, rather than online.

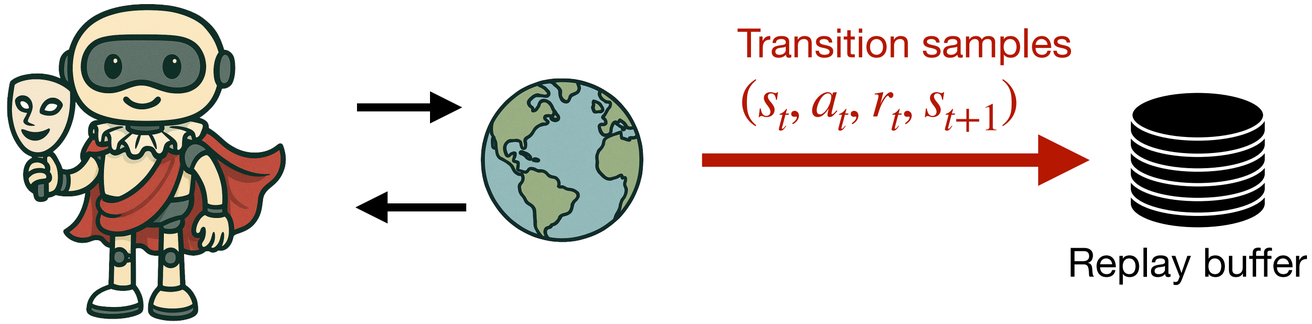
As in DQN, we used a replay buffer to address these issues. The replay buffer is a finite sized cache $\mathcal{R}$. Transitions were sampled from the environment according to the exploration policy and the tuple (s_t, a_t, r_t, s_{t+1}) was stored in the replay buffer. When the replay buffer was full the oldest samples were discarded. At each timestep the actor and critic are updated by sampling a minibatch uniformly from the buffer. Because DDPG is an off-policy algorithm, the replay buffer can be large, allowing the algorithm to benefit from learning across a set of uncorrelated transitions.

Experience replay in DDPG

Why? Alleviate problems of correlated data and non-stationary distributions.
Leverage the strengths of off-policy learning: We can learn from past data collected by different, behaviour policies.

One challenge when using neural networks for reinforcement learning is that most optimization algorithms assume that the samples are independently and identically distributed. Obviously, when the samples are generated from exploring sequentially in an environment this assumption no longer holds. Additionally, to make efficient use of hardware optimizations, it is essential to learn in mini-batches, rather than online.

As in DQN, we used a replay buffer to address these issues. The replay buffer is a finite sized cache $\mathcal{R}$. Transitions were sampled from the environment according to the exploration policy and the tuple (s_t, a_t, r_t, s_{t+1}) was stored in the replay buffer. When the replay buffer was full the oldest samples were discarded. At each timestep the actor and critic are updated by sampling a minibatch uniformly from the buffer. Because DDPG is an off-policy algorithm, the replay buffer can be large, allowing the algorithm to benefit from learning across a set of uncorrelated transitions.



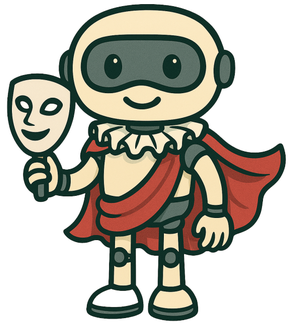
Data collection

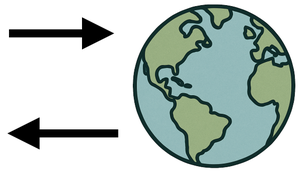
Experience replay in DDPG

Why? Alleviate problems of correlated data and non-stationary distributions.
Leverage the strengths of off-policy learning: We can learn from past data collected by different, behaviour policies.

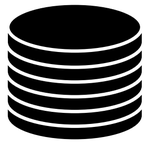
One challenge when using neural networks for reinforcement learning is that most optimization algorithms assume that the samples are independently and identically distributed. Obviously, when the samples are generated from exploring sequentially in an environment this assumption no longer holds. Additionally, to make efficient use of hardware optimizations, it is essential to learn in mini-batches, rather than online.

As in DQN, we used a replay buffer to address these issues. The replay buffer is a finite sized cache $\mathcal{R}$. Transitions were sampled from the environment according to the exploration policy and the tuple (s_t, a_t, r_t, s_{t+1}) was stored in the replay buffer. When the replay buffer was full the oldest samples were discarded. At each timestep the actor and critic are updated by sampling a minibatch uniformly from the buffer. Because DDPG is an off-policy algorithm, the replay buffer can be large, allowing the algorithm to benefit from learning across a set of uncorrelated transitions.





Transition samples
(s_t, a_t, r_t, s_{t+1})



Replay buffer

Data minibatch
$\{(s_t^{(i)}, a_t^{(i)}, r_t^{(i)}, s_{t+1}^{(i)})\}_{i=1}^{N}$

$$\nabla_\theta J(\theta) = \mathbb{E}_{s_t \sim \beta}\left[\nabla_\theta Q_w(s,a)\,|_{s=s_t, a=\mu_\theta(s_t)}\right]$$

$$L(w) = \mathbb{E}_{s_t \sim \rho^\beta, a_t \sim \beta(\cdot|s_t)}\left[\left(Q_w(s_t,a_t) - y_t\right)^2\right]$$

Data collection

Training

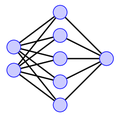
DDPG actor-critic networks with “targets”

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

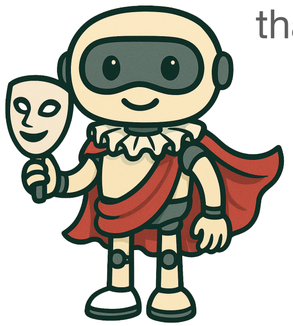
DDPG actor-critic networks with “targets”

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

Deterministic actor network $a = \mu_\theta(s)$



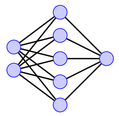
This is the “online” policy that we are trying to improve.



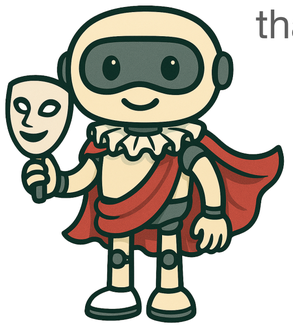
DDPG actor-critic networks with “targets”

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

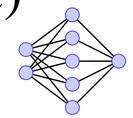
Deterministic actor network $a = \mu_\theta(s)$

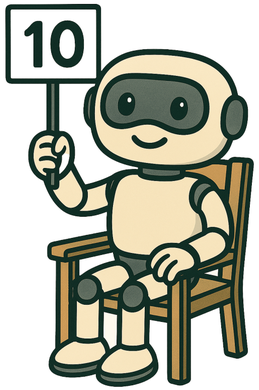


This is the “online” policy that we are trying to improve.



Parameterized critic network $Q_w(s, a)$



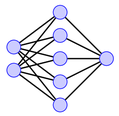


The “online” Q-function that we train to evaluate the actor.

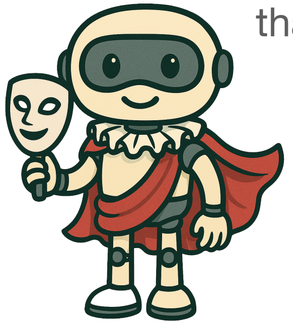
DDPG actor-critic networks with “targets”

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

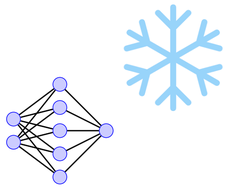
Deterministic actor network $a = \mu_\theta(s)$



This is the “online” policy that we are trying to improve.

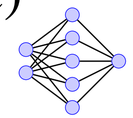


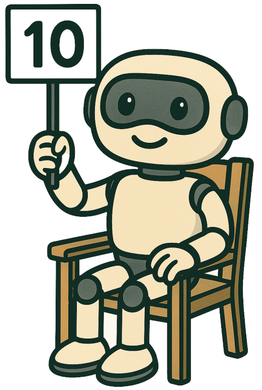
The target actor network $a = \bar{\mu}_\theta(s)$



A frozen, or slowly-updating copy of the actor network.

Parameterized critic network $Q_w(s, a)$



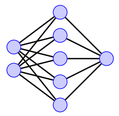


The “online” Q-function that we train to evaluate the actor.

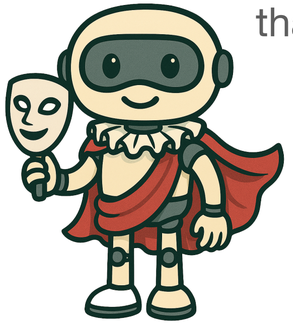
DDPG actor-critic networks with “targets”

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

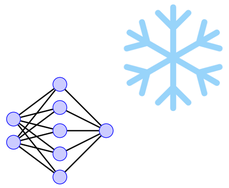
Deterministic actor network $a = \mu_\theta(s)$



This is the “online” policy that we are trying to improve.

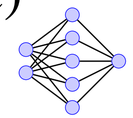


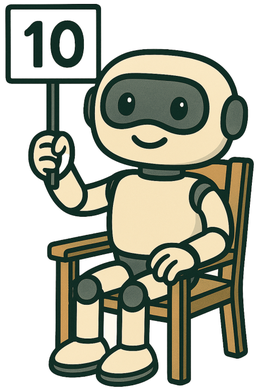
The target actor network $a = \bar{\mu}_\theta(s)$



A frozen, or slowly-updating copy of the actor network.

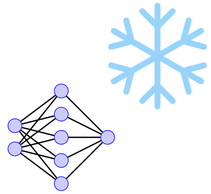
Parameterized critic network $Q_w(s, a)$





The “online” Q-function that we train to evaluate the actor.

The target critic network $\bar{Q}_w(s, a)$

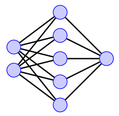


A frozen, or slowly-updating copy of the critic network.

DDPG actor-critic networks with “targets”

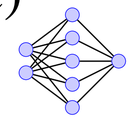
Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using “soft” target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

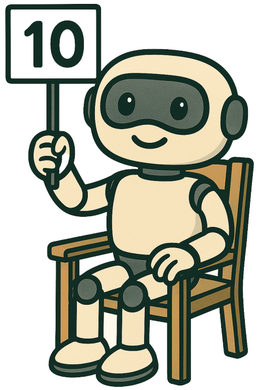
Deterministic actor network $a = \mu_\theta(s)$



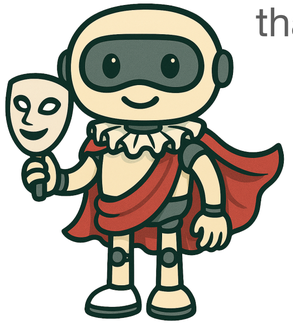
This is the “online” policy that we are trying to improve.

Parameterized critic network $Q_w(s, a)$

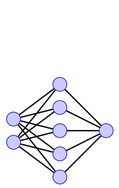




The “online” Q-function that we train to evaluate the actor.



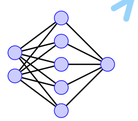
The target actor network $a = \bar{\mu}_\theta(s)$



A frozen, or slowly-updating copy of the actor network.

Target networks used to create the Q-learning target.

The target critic network $\bar{Q}_w(s, a)$



A frozen, or slowly-updating copy of the critic network.

Updating the target networks: soft updates

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using "soft" target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

DQN does "hard" updates: copy the online network weights to the target network weights every N steps.

Updating the target networks: soft updates

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using "soft" target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

DQN does "hard" updates: copy the online network weights to the target network weights every N steps.

DDPG instead introduce "Polyak averaging".

After every training step, slowly mix the online weights with the previous target weights as follows:

$$\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}$$ Target actor weight update

$$\bar{w} \leftarrow \tau w + (1 - \tau)\bar{w}$$ Target critic weight update

τ is typically small, e.g., 0.001 in DDPG paper.

Updating the target networks: soft updates

Directly implementing Q learning (equation 4) with neural networks proved to be unstable in many environments. Since the network $Q(s, a|\theta^Q)$ being updated is also used in calculating the target value (equation 5), the Q update is prone to divergence. Our solution is similar to the target network used in (Mnih et al., 2013) but modified for actor-critic and using "soft" target updates, rather than directly copying the weights. We create a copy of the actor and critic networks, $Q'(s, a|\theta^{Q'})$ and $\mu'(s|\theta^{\mu'})$ respectively, that are used for calculating the target values. The weights of these target networks are then updated by having them slowly track the learned networks: $\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$ with $\tau \ll 1$. This means that the target values are constrained to change slowly, greatly improving the stability of learning. This simple change moves the relatively unstable problem of learning the action-value function closer to the case of supervised learning, a problem for which robust solutions exist. We found that having both a target μ' and Q' was required to have stable targets y_i in order to consistently train the critic without divergence. This may slow learning, since the target network delays the propagation of value estimations. However, in practice we found this was greatly outweighed by the stability of learning.

DQN does "hard" updates: copy the online network weights to the target network weights every N steps.

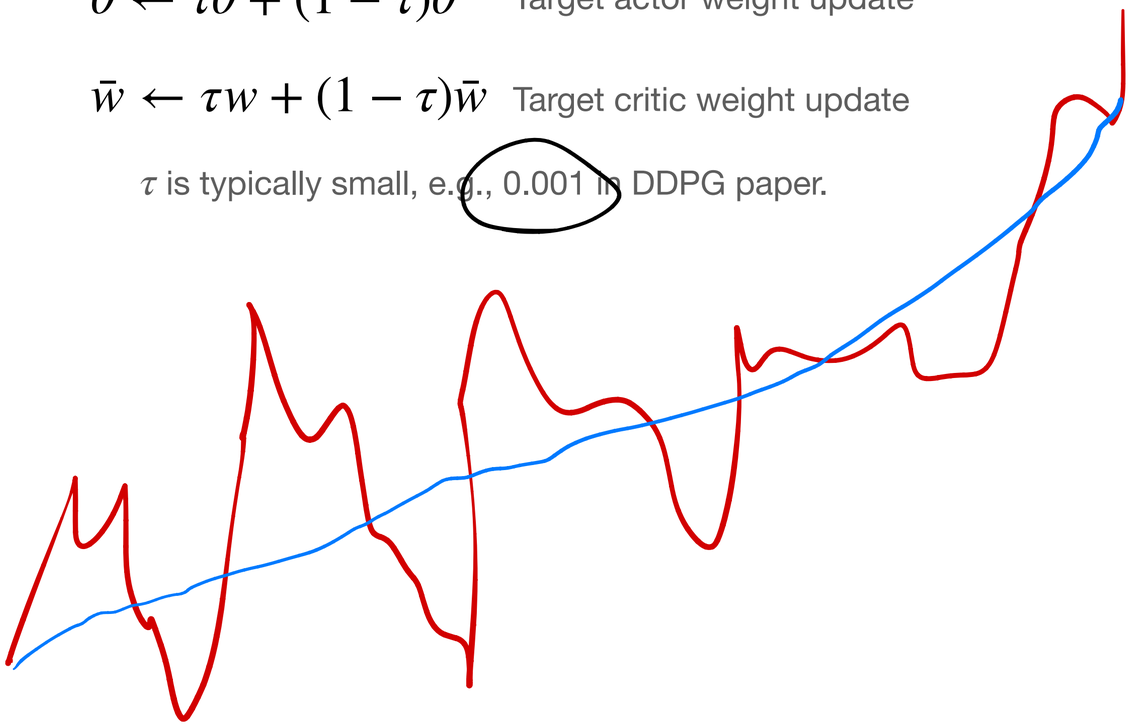
DDPG instead introduce "Polyak averaging".

After every training step, slowly mix the online weights with the previous target weights as follows:

$$\bar{\theta} \leftarrow \tau\theta + (1 - \tau)\bar{\theta}$$ Target actor weight update

$$\bar{w} \leftarrow \tau w + (1 - \tau)\bar{w}$$ Target critic weight update

τ is typically small, e.g., 0.001 in DDPG paper.



Important stabilization trick in RL.

- The target networks create the TD target y_i.
- In DDPG, both actor and critic are being updated -> lanscape is constantly shifting. Hard updates would result in drastic jumps every step.
- "Soft updating" acts like a low-pass filter, making $\bar{\mu}_\theta$ and $\bar{Q}_w$ update more slowly and smoothly.

Batch normalization and exploration through added noise to the actions

When learning from low dimensional feature vector observations, the different components of the observation may have different physical units (for example, positions versus velocities) and the ranges may vary across environments. This can make it difficult for the network to learn effectively and may make it difficult to find hyper-parameters which generalise across environments with different scales of state values.

One approach to this problem is to manually scale the features so they are in similar ranges across environments and units. We address this issue by adapting a recent technique from deep learning called *batch normalization* (Ioffe & Szegedy, 2015). This technique normalizes each dimension across the samples in a minibatch to have unit mean and variance. In addition, it maintains a running average of the mean and variance to use for normalization during testing (in our case, during exploration or evaluation). In deep networks, it is used to minimize covariance shift during training, by ensuring that each layer receives whitened input. In the low-dimensional case, we used batch normalization on the state input and all layers of the μ network and all layers of the Q network prior to the action input (details of the networks are given in the supplementary material). With batch normalization, we were able to learn effectively across many different tasks with differing types of units, without needing to manually ensure the units were within a set range.

Batch normalization to handle different scales of the different dimensions of the action space.

Batch normalization and exploration through added noise to the actions

When learning from low dimensional feature vector observations, the different components of the observation may have different physical units (for example, positions versus velocities) and the ranges may vary across environments. This can make it difficult for the network to learn effectively and may make it difficult to find hyper-parameters which generalise across environments with different scales of state values.

One approach to this problem is to manually scale the features so they are in similar ranges across environments and units. We address this issue by adapting a recent technique from deep learning called *batch normalization* (Ioffe & Szegedy, 2015). This technique normalizes each dimension across the samples in a minibatch to have unit mean and variance. In addition, it maintains a running average of the mean and variance to use for normalization during testing (in our case, during exploration or evaluation). In deep networks, it is used to minimize covariance shift during training, by ensuring that each layer receives whitened input. In the low-dimensional case, we used batch normalization on the state input and all layers of the μ network and all layers of the Q network prior to the action input (details of the networks are given in the supplementary material). With batch normalization, we were able to learn effectively across many different tasks with differing types of units, without needing to manually ensure the units were within a set range.

Batch normalization to handle different scales of the different dimensions of the action space.

Adding exploration noise

A major challenge of learning in continuous action spaces is exploration. An advantage of off-policies algorithms such as DDPG is that we can treat the problem of exploration independently from the learning algorithm. We constructed an exploration policy μ' by adding noise sampled from a noise process $\mathcal{N}$ to our actor policy

$$\mu'(s_t) = \mu(s_t|\theta_t^\mu) + \mathcal{N} \tag{7}$$

$\mathcal{N}$ can be chosen to suit the environment. As detailed in the supplementary materials we used an Ornstein-Uhlenbeck process (Uhlenbeck & Ornstein, 1930) to generate temporally correlated exploration for exploration efficiency in physical control problems with inertia (similar use of auto-correlated noise was introduced in (Wawrzyński, 2015)).

Putting it all together: the full DDPG algorithm

Algorithm 1 DDPG algorithm

Randomly initialize critic network $Q(s, a|\theta^Q)$ and actor $\mu(s|\theta^\mu)$ with weights θ^Q and θ^μ.
Initialize target network Q' and μ' with weights $\theta^{Q'} \leftarrow \theta^Q$, $\theta^{\mu'} \leftarrow \theta^\mu$
Initialize replay buffer R
for episode = 1, M **do**
  Initialize a random process $\mathcal{N}$ for action exploration
  Receive initial observation state s_1
  for t = 1, T **do**
    Select action $a_t = \mu(s_t|\theta^\mu) + \mathcal{N}_t$ according to the current policy and exploration noise
    Execute action a_t and observe reward r_t and observe new state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in R
    Sample a random minibatch of N transitions (s_i, a_i, r_i, s_{i+1}) from R
    Set $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$
    Update critic by minimizing the loss: $L = \frac{1}{N}\sum_i (y_i - Q(s_i, a_i|\theta^Q))^2$
    Update the actor policy using the sampled policy gradient:

$$\nabla_{\theta^\mu} J \approx \frac{1}{N} \sum_i \nabla_a Q(s, a|\theta^Q)|_{s=s_i, a=\mu(s_i)} \nabla_{\theta^\mu} \mu(s|\theta^\mu)|_{s_i}$$

    Update the target networks:

$$\theta^{Q'} \leftarrow \tau\theta^Q + (1-\tau)\theta^{Q'}$$
$$\theta^{\mu'} \leftarrow \tau\theta^\mu + (1-\tau)\theta^{\mu'}$$

  end for
end for

initializing actor, critic, target networks

sampling exploration noise

adding to minibuffer

sampling minibatch

use target networks to construct TD error

use DPG theorem to improve actor

soft target updates.

DDPG experimental results

Figure 1: Example screenshots of a sample of environments we attempt to solve with DDPG. In order from the left: the cartpole swing-up task, a reaching task, a gasp and move task, a puck-hitting task, a monoped balancing task, two locomotion tasks and Torcs (driving simulator). We tackle all tasks using both low-dimensional feature vector and high-dimensional pixel inputs. Detailed descriptions of the environments are provided in the supplementary. Movies of some of the learned policies are available at `https://goo.gl/J4PIAz`.

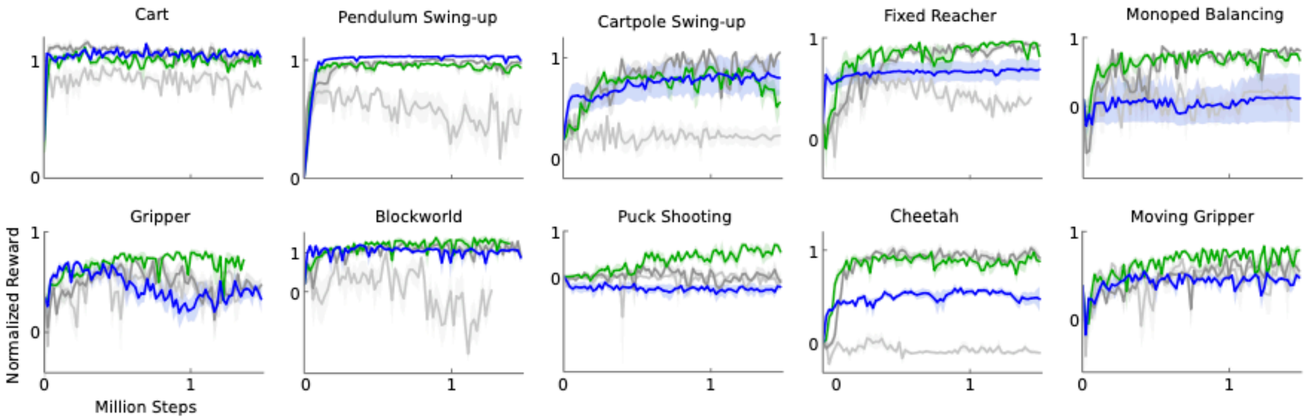


Figure 2: Performance curves for a selection of domains using variants of DPG: original DPG algorithm (minibatch NFQCA) with batch normalization (light grey), with target network (dark grey), with target networks and batch normalization (green), with target networks from pixel-only inputs (blue). Target networks are crucial.

Table 1: Performance after training across all environments for at most 2.5 million steps. We report both the average and best observed (across 5 runs). All scores, except Torcs, are normalized so that a random agent receives 0 and a planning algorithm 1; for Torcs we present the raw reward score. We include results from the DDPG algorithn in the low-dimensional (*lowd*) version of the environment and high-dimensional (*pix*). For comparision we also include results from the original DPG algorithm with a replay buffer and batch normalization (*cntrl*).

environment	$R_{av,lowd}$	$R_{best,lowd}$	$R_{av,pix}$	$R_{best,pix}$	$R_{av,cntrl}$	$R_{best,cntrl}$
blockworld1	1.156	1.511	0.466	1.299	-0.080	1.260
blockworld3da	0.340	0.705	0.889	2.225	-0.139	0.658
canada	0.303	1.735	0.176	0.688	0.125	1.157
canada2d	0.400	0.978	-0.285	0.119	-0.045	0.701
cart	0.938	1.336	1.096	1.258	0.343	1.216
cartpole	0.844	1.115	0.482	1.138	0.244	0.755
cartpoleBalance	0.951	1.000	0.335	0.996	-0.468	0.528
cartpoleParallelDouble	0.549	0.900	0.188	0.323	0.197	0.572
cartpoleSerialDouble	0.272	0.719	0.195	0.642	0.143	0.701
cartpoleSerialTriple	0.736	0.946	0.412	0.427	0.583	0.942
cheetah	0.903	1.206	0.457	0.792	-0.008	0.425
fixedReacher	0.849	1.021	0.693	0.981	0.259	0.927
fixedReacherDouble	0.924	0.996	0.872	0.943	0.290	0.995
fixedReacherSingle	0.954	1.000	0.827	0.995	0.620	0.999
gripper	0.655	0.972	0.406	0.790	0.461	0.816
gripperRandom	0.618	0.937	0.082	0.791	0.557	0.808
hardCheetah	1.311	1.990	1.204	1.431	-0.031	1.411
hopper	0.676	0.936	0.112	0.924	0.078	0.917
hyq	0.416	0.722	0.234	0.672	0.198	0.618
movingGripper	0.474	0.936	0.480	0.644	0.416	0.805
pendulum	0.946	1.021	0.663	1.055	0.099	0.951
reacher	0.720	0.987	0.194	0.878	0.231	0.953
reacher3daFixedTarget	0.585	0.943	0.453	0.922	0.204	0.631
reacher3daRandomTarget	0.467	0.739	0.374	0.735	-0.046	0.158
reacherSingle	0.981	1.102	1.000	1.083	1.010	1.083
walker2d	0.705	1.573	0.944	1.476	0.393	1.397
torcs	-393.385	1840.036	-401.911	1876.284	-911.034	1961.600

Key messages: DDPG works, core components are crucial for stability, it can learn end-to-end from pixels, and it can outperform baselines.
Also, results are sensitive to batch normalization.

So… did we solve RL yet?

No! DDPG is quite brittle in practice.

- Deadly triad ⟶ overestimation bias.
  - ↳ $\max_{a'} Q(s,a)$ and Q has errors, those errors are amplified.

Recall: Maximization bias in (tabular) Q-learning

Q-Learning update: $$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$

Update rule moves current estimate $\hat{Q}(s, a)$ towards the target $r_t + \gamma \max_{a \in A} \hat{Q}(s, a)$.

Note: Our target is defined using the current estimate $\hat{Q}(s, a)$, instead of the true optimal value function $Q^*(s, a)$.

Recall: Maximization bias in (tabular) Q-learning

Q-Learning update: $$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$

Update rule moves current estimate $\hat{Q}(s, a)$ towards the target $r_t + \gamma \max_{a \in A} \hat{Q}(s, a)$.

Note: Our target is defined using the current estimate $\hat{Q}(s, a)$, instead of the true optimal value function $Q^*(s, a)$.

This introduces **maximization bias**: We are taking a maximization over *noisy* estimates of the value.
This can bias the results to be higher than they should be, especially early on in training!

Recall: Maximization bias in (tabular) Q-learning

Q-Learning update: $$\hat{Q}(s_t, a_t) \leftarrow \hat{Q}(s_t, a_t) + \alpha \left(r_t + \gamma \max_{a \in A} \hat{Q}(s_{t+1}, a) - \hat{Q}(s_t, a_t) \right)$$

Update rule moves current estimate $\hat{Q}(s, a)$ towards the target $r_t + \gamma \max_{a \in A} \hat{Q}(s, a)$.

Note: Our target is defined using the current estimate $\hat{Q}(s, a)$, instead of the true optimal value function $Q^*(s, a)$.

This introduces **maximization bias**: We are taking a maximization over *noisy* estimates of the value.
This can bias the results to be higher than they should be, especially early on in training!

Intuition: If you take several noisy estimates of numbers, the **maximum** of those estimates is, on average, a little too high. This is because you're likely to pick the one that got lucky noise in the positive direction.

Recall: Maximization bias in (tabular) Q-learning

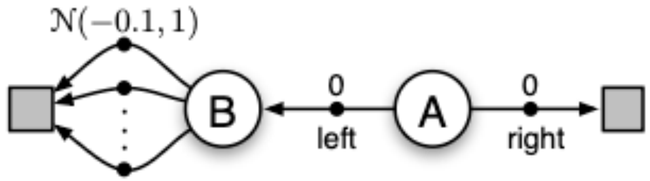


Figure courtesy of Sutton and Barto (2018) pg. 135.

- Initial state A.
- Two actions from A, left or right.
  - Right leads to terminal state with no reward.
  - Left leads to state B.
- From state B, there are MANY available actions, with rewards distributed according to $R \sim \mathcal{N}(-0.1, 1)$.
- So, the expected value of taking action *left* and reaching state B is -0.1, which is lower than the expected value of taking *right* and terminating with reward of 0.0.

Recall: Maximization bias in (tabular) Q-learning

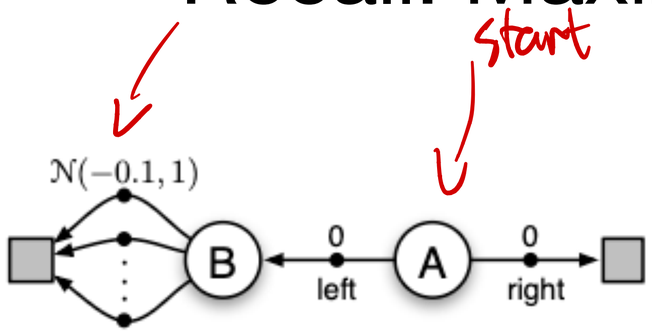
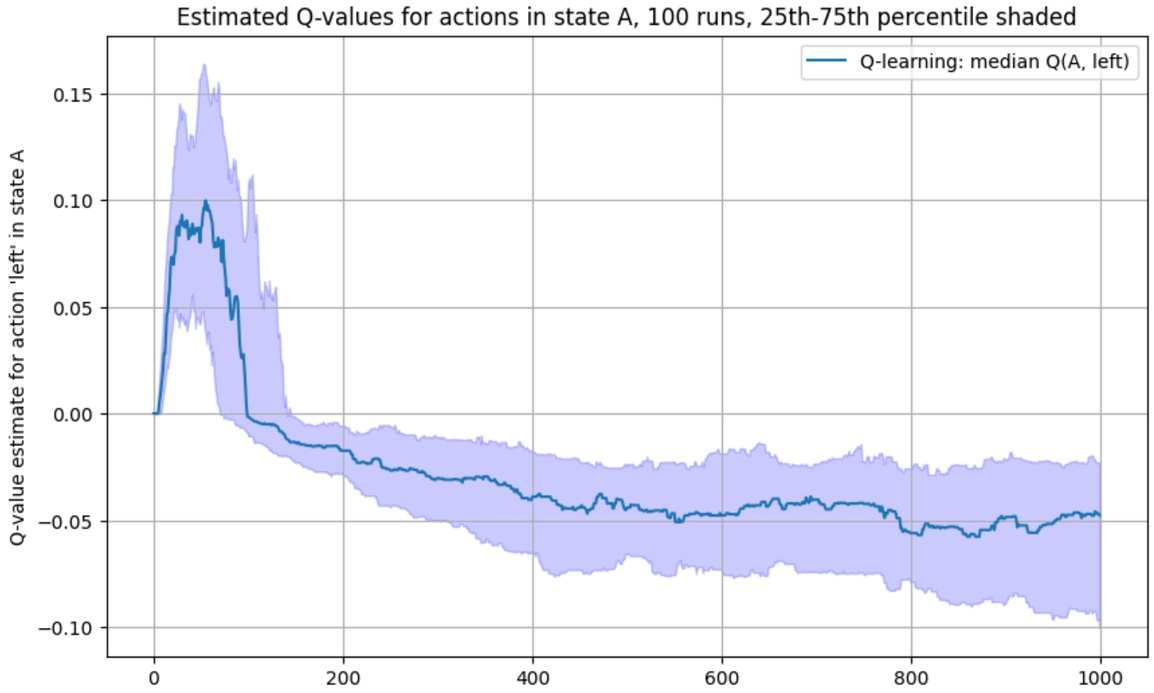
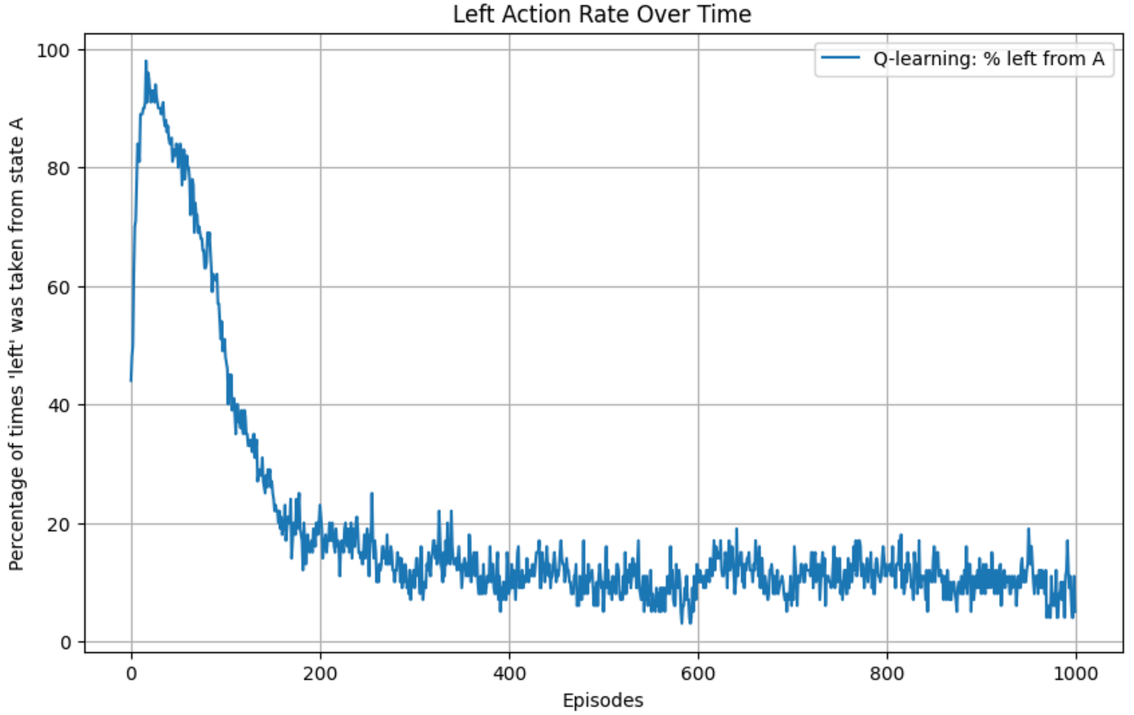


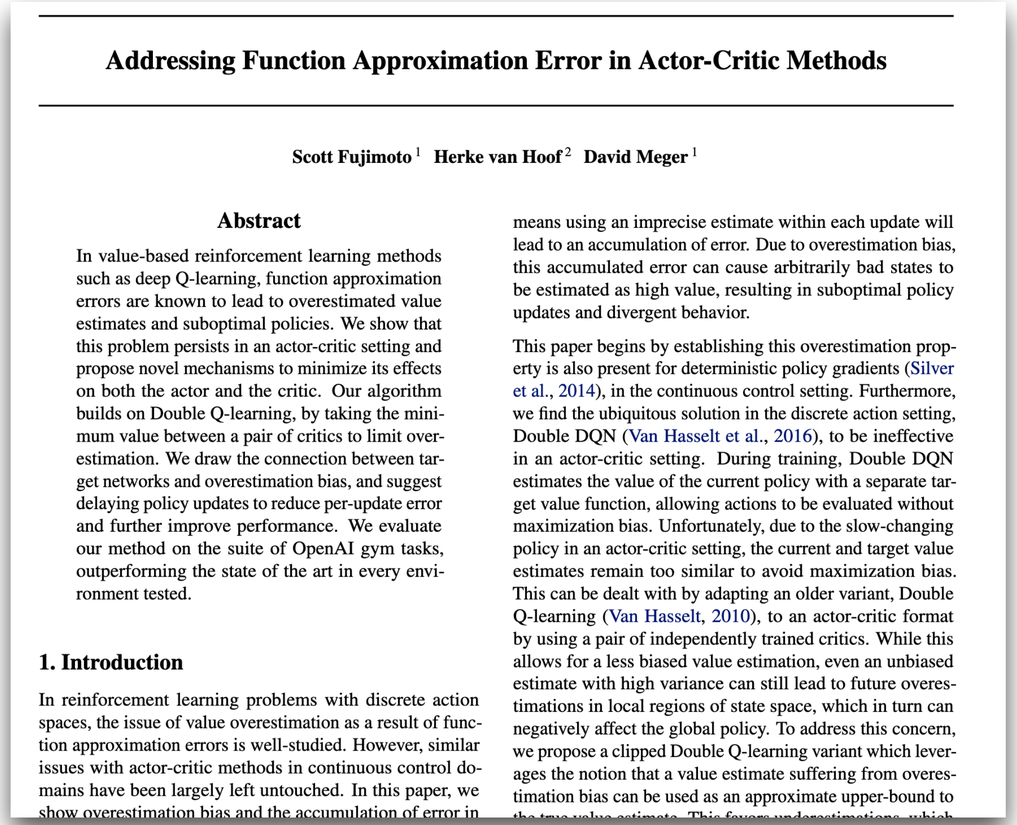
Figure courtesy of Sutton and Barto (2018) pg. 135.

- Initial state A.
- Two actions from A, left or right.
  - Right leads to terminal state with no reward.
  - Left leads to state B.
- From state B, there are MANY available actions, with rewards distributed according to $R \sim \mathcal{N}(-0.1, 1)$.
- So, the expected value of taking action *left* and reaching state B is -0.1, which is lower than the expected value of taking *right* and terminating with reward of 0.0.





Twin-delayed deep deterministic policy gradient algorithms (TD3)



Addressing Function Approximation Error in Actor-Critic Methods

Scott Fujimoto [1] **Herke van Hoof** [2] **David Meger** [1]

Abstract

In value-based reinforcement learning methods such as deep Q-learning, function approximation errors are known to lead to overestimated value estimates and suboptimal policies. We show that this problem persists in an actor-critic setting and propose novel mechanisms to minimize its effects on both the actor and the critic. Our algorithm builds on Double Q-learning, by taking the minimum value between a pair of critics to limit overestimation. We draw the connection between target networks and overestimation bias, and suggest delaying policy updates to reduce per-update error and further improve performance. We evaluate our method on the suite of OpenAI gym tasks, outperforming the state of the art in every environment tested.

1. Introduction

In reinforcement learning problems with discrete action spaces, the issue of value overestimation as a result of function approximation errors is well-studied. However, similar issues with actor-critic methods in continuous control domains have been largely left untouched. In this paper, we show overestimation bias and the accumulation of error in means using an imprecise estimate within each update will lead to an accumulation of error. Due to overestimation bias, this accumulated error can cause arbitrarily bad states to be estimated as high value, resulting in suboptimal policy updates and divergent behavior.

This paper begins by establishing this overestimation property is also present for deterministic policy gradients (Silver et al., 2014), in the continuous control setting. Furthermore, we find the ubiquitous solution in the discrete action setting, Double DQN (Van Hasselt et al., 2016), to be ineffective in an actor-critic setting. During training, Double DQN estimates the value of the current policy with a separate target value function, allowing actions to be evaluated without maximization bias. Unfortunately, due to the slow-changing policy in an actor-critic setting, the current and target value estimates remain too similar to avoid maximization bias. This can be dealt with by adapting an older variant, Double Q-learning (Van Hasselt, 2010), to an actor-critic format by using a pair of independently trained critics. While this allows for a less biased value estimation, even an unbiased estimate with high variance can still lead to future overestimations in local regions of state space, which in turn can negatively affect the global policy. To address this concern, we propose a clipped Double Q-learning variant which leverages the notion that a value estimate suffering from overestimation bias can be used as an approximate upper-bound to

Key idea: Address the key stability issues with DDPG, with several well-designed engineering fixes.

1. Clipped Double Q-Learning (the “Twin”) for Q-value overestimation.
2. Delayed policy updates to limit propagation of TD errors.
3. Target policy smoothing.

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Then…

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{approx}}}(s, \mu_{\theta_{approx}})\right]$$

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Then…

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{approx}}}(s, \mu_{\theta_{approx}})\right]$$

The approximate policy optimizes the approximate value estimate, so, under that approximate value estimate it must be higher than the “true” policy update.

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Then…

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{approx}}}(s, \mu_{\theta_{approx}})\right]$$

The approximate policy optimizes the approximate value estimate, so, under that approximate value estimate it must be higher than the “true” policy update.

The ground truth value of the true policy is higher than the ground truth value of the flawed, approximate policy.

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Then…

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{approx}}}(s, \mu_{\theta_{approx}})\right]$$

The approximate policy optimizes the approximate value estimate, so, under that approximate value estimate it must be higher than the “true” policy update.

Assumption.

The ground truth value of the true policy is higher than the ground truth value of the flawed, approximate policy.

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Then…

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{approx}}}(s, \mu_{\theta_{approx}})\right]$$

The approximate policy optimizes the approximate value estimate, so, under that approximate value estimate it must be higher than the “true” policy update.

Assumption.

The ground truth value of the true policy is higher than the ground truth value of the flawed, approximate policy.

So, if in expectation the value estimate (of the true policy $\mu_{\theta_{true}}$) is at least as large as the true value $\mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right]$, then the value estimate of the approximate policy will be overestimated:

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{approx}})\right]$$

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Then…

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{approx}}}(s, \mu_{\theta_{approx}})\right]$$

The approximate policy optimizes the approximate value estimate, so, under that approximate value estimate it must be higher than the “true” policy update.

Assumption.

The ground truth value of the true policy is higher than the ground truth value of the flawed, approximate policy.

So, if in expectation the value estimate (of the true policy $\mu_{\theta_{true}}$) is at least as large as the true value $\mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right]$, then the value estimate of the approximate policy will be overestimated:

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{approx}})\right]$$

i.e. if the value is overestimated, then the actor’s DPG update mechanism will transfer this error to the policy we are learning.

Overestimation bias in actor-critic methods

Let $\theta_{approx} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q_w(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the approximate critic $Q_w(s,a)$.

And $\theta_{true} = \theta + \alpha \mathbb{E}_{s \sim \rho^{\mu_\theta}} \left[\nabla_\theta \mu_\theta(s) \nabla_a Q^{\mu_\theta}(s,a) \mid_{a=\mu_\theta(s)} \right]$ be the actor update using the TRUE critic $Q^{\mu_\theta}(s,a)$.

Then…

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{approx}}}(s, \mu_{\theta_{approx}})\right]$$

The approximate policy optimizes the approximate value estimate, so, under that approximate value estimate it must be higher than the “true” policy update.

Assumption.

The ground truth value of the true policy is higher than the ground truth value of the flawed, approximate policy.

So, if in expectation the value estimate (of the true policy $\mu_{\theta_{true}}$) is at least as large as the true value $\mathbb{E}\left[Q_w(s, \mu_{\theta_{true}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{true}})\right]$, then the value estimate of the approximate policy will be overestimated:

$$\mathbb{E}\left[Q_w(s, \mu_{\theta_{approx}})\right] \geq \mathbb{E}\left[Q^{\mu_{\theta_{true}}}(s, \mu_{\theta_{approx}})\right]$$

i.e. if the value is overestimated, then the actor’s DPG update mechanism will transfer this error to the policy we are learning.

> Although this overestimation may be minimal with each update, the presence of error raises two concerns. Firstly, the overestimation may develop into a more significant bias over many updates if left unchecked. Secondly, an inaccurate value estimate may lead to poor policy updates. This is particularly problematic because a feedback loop is created, in which suboptimal actions might be highly rated by the suboptimal critic, reinforcing the suboptimal action in the next policy update.

Recall: countering maximization bias via double Q-Learning

Intuition: maximization bias comes from using the same $\hat{Q}(s,a)$ to **select the best action**, and to **evaluate its Q-value**. If the *same noise* appears in both roles, it reinforces itself.

Select the best action

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}(s', \text{argmax}_{a'} \hat{Q}(s',a')) - \hat{Q}(s,a) \right]$$

Estimate the value of this selection

Recall: countering maximization bias via double Q-Learning

Intuition: maximization bias comes from using the same $\hat{Q}(s,a)$ to **select the best action**, and to **evaluate its Q-value**. If the *same noise* appears in both roles, it reinforces itself.

Select the best action

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}(s', \text{argmax}_{a'} \hat{Q}(s',a')) - \hat{Q}(s,a) \right]$$

Estimate the value of this selection

Solution: maintain two estimates $\hat{Q}_A(s,a)$ and $\hat{Q}_B(s,a)$. Each update, randomly select one of the estimates to use for action selection, and the other for action evaluation.

Recall: countering maximization bias via double Q-Learning

Intuition: maximization bias comes from using the same $\hat{Q}(s,a)$ to **select the best action**, and to **evaluate its Q-value**. If the *same noise* appears in both roles, it reinforces itself.

Select the best action

$$\hat{Q}(s,a) \leftarrow \hat{Q}(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}(s', \text{argmax}_{a'} \hat{Q}(s',a')) - \hat{Q}(s,a) \right]$$

Estimate the value of this selection

Solution: maintain two estimates $\hat{Q}_A(s,a)$ and $\hat{Q}_B(s,a)$. Each update, randomly select one of the estimates to use for action selection, and the other for action evaluation.

That is, each update, choose A or B with 50/50 chance.

If updating A:

$$\hat{Q}_A(s,a) \leftarrow \hat{Q}_A(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}_B(s', \text{argmax}_{a'} \hat{Q}_A(s',a')) - \hat{Q}_A(s,a) \right]$$

If updating B:

$$\hat{Q}_B(s,a) \leftarrow \hat{Q}_B(s,a) + \alpha \left[R(s,a) + \gamma \hat{Q}_A(s', \text{argmax}_{a'} \hat{Q}_B(s',a')) - \hat{Q}_B(s,a) \right]$$

Recall: countering maximization bias via double Q-Learning

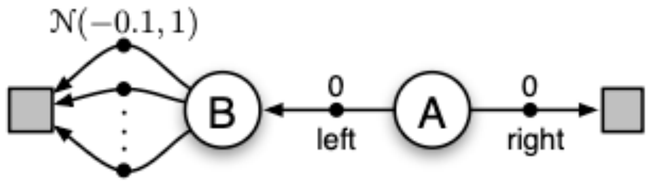


Figure courtesy of Sutton and Barto (2018) pg. 135.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\textit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \arg\max_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \arg\max_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal

Recall: countering maximization bias via double Q-Learning

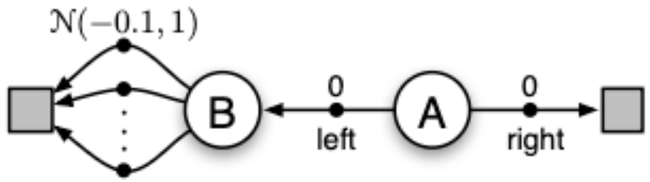


Figure courtesy of Sutton and Barto (2018) pg. 135.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\textit{terminal}, \cdot) = 0$

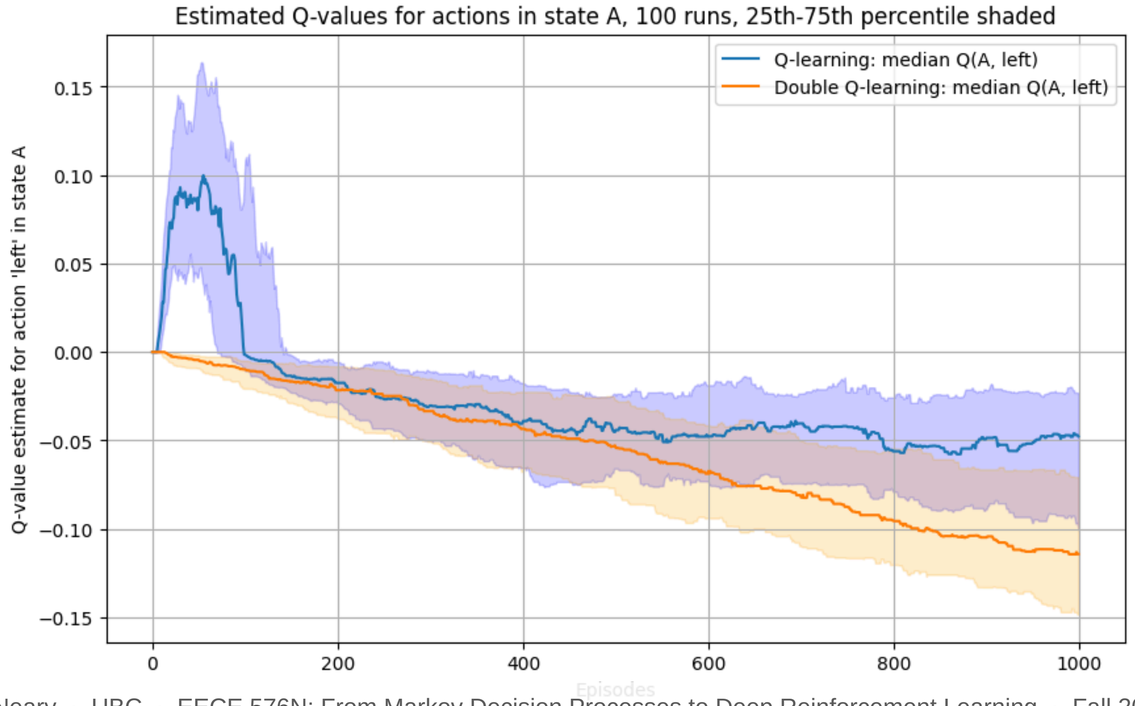
Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:

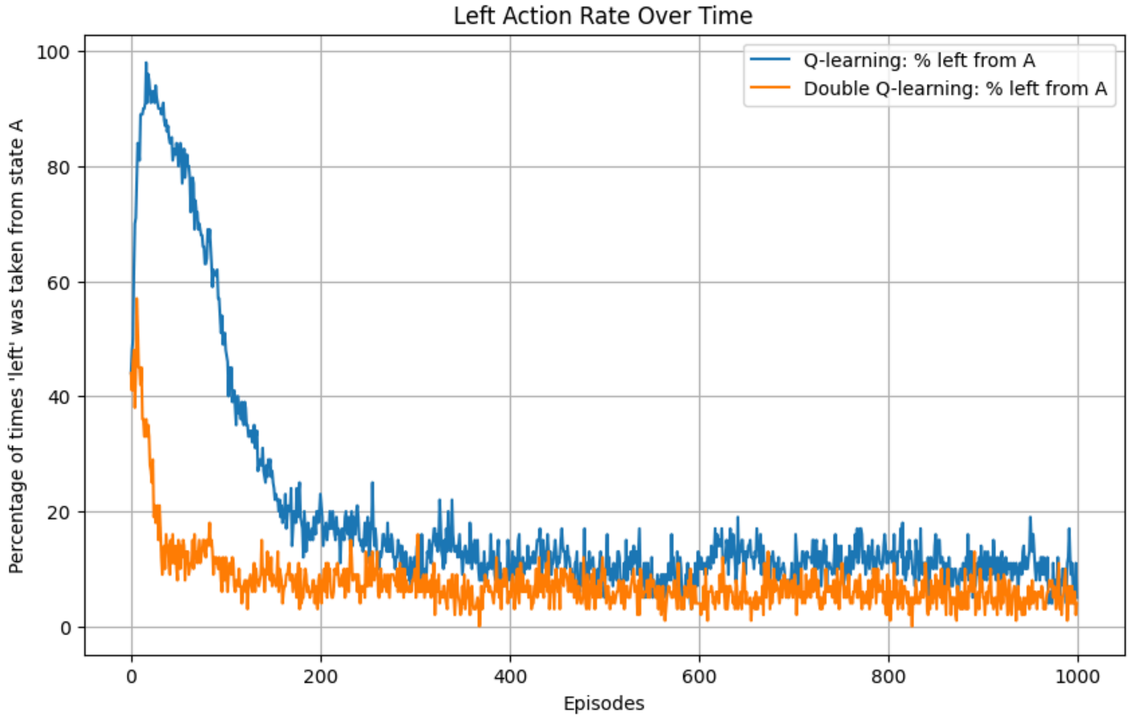
$$Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \arg\max_a Q_1(S', a)\big) - Q_1(S, A)\Big)$$

    else:

$$Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \arg\max_a Q_2(S', a)\big) - Q_2(S, A)\Big)$$

    $S \leftarrow S'$
  until S is terminal





The solution: Clipped double Q-learning for actor-critic

Double Q-learning:

Introduce a pair of critics $Q_{w_1}(s,a)$ and $Q_{w_2}(s,a)$, as well as a pair of actors $\mu_{\theta_1}(s)$ and $\mu_{\theta_2}(s)$ (and all of their corresponding target networks).

Where $\mu_{\theta_1}(s)$ is optimized w.r.t. $Q_{w_1}(s,a)$ and $\mu_{\theta_2}(s)$ is optimized w.r.t. $Q_{w_2}(s,a)$.

$$y_1 = r + \gamma \bar{Q}_{w_1}(s, \bar{\mu}_{\theta_2}(s))$$

The solution: Clipped double Q-learning for actor-critic

Double Q-learning:

Introduce a pair of critics $Q_{w_1}(s,a)$ and $Q_{w_2}(s,a)$, as well as a pair of actors $\mu_{\theta_1}(s)$ and $\mu_{\theta_2}(s)$ (and all of their corresponding target networks).

Where $\mu_{\theta_1}(s)$ is optimized w.r.t. $Q_{w_1}(s,a)$ and $\mu_{\theta_2}(s)$ is optimized w.r.t. $Q_{w_2}(s,a)$.

$$y_1 = r + \gamma \bar{Q}_{w_1}(s, \bar{\mu}_{\theta_2}(s))$$

$$y_2 = r + \gamma \bar{Q}_{w_2}(s, \bar{\mu}_{\theta_1}(s))$$

Clipped double Q-learning:

$$y = \min_{i=1,2} y_i \longrightarrow L(w_i) = \mathbb{E}\left[\left(Q_{w_i}(s,a) - y\right)^2\right]$$

The solution: Clipped double Q-learning for actor-critic

Double Q-learning:

Introduce a pair of critics $Q_{w_1}(s,a)$ and $Q_{w_2}(s,a)$, as well as a pair of actors $\mu_{\theta_1}(s)$ and $\mu_{\theta_2}(s)$ (and all of their corresponding target networks).

Where $\mu_{\theta_1}(s)$ is optimized w.r.t. $Q_{w_1}(s,a)$ and $\mu_{\theta_2}(s)$ is optimized w.r.t. $Q_{w_2}(s,a)$.

$$y_1 = r + \gamma \bar{Q}_{w_1}(s, \bar{\mu}_{\theta_2}(s))$$

$$y_2 = r + \gamma \bar{Q}_{w_2}(s, \bar{\mu}_{\theta_1}(s))$$

Clipped double Q-learning:

$$y = \min_{i=1,2} y_i \longrightarrow L(w_i) = \mathbb{E}\left[\left(Q_{w_i}(s,a) - y\right)^2\right]$$

With Clipped Double Q-learning, the value target cannot introduce any additional overestimation over using the standard Q-learning target. While this update rule may induce an underestimation bias, this is far preferable to overestimation bias, as unlike overestimated actions, the value of underestimated actions will not be explicitly propagated through the policy update.

The solution: Clipped double Q-learning for actor-critic

Double Q-learning:

Introduce a pair of critics $Q_{w_1}(s,a)$ and $Q_{w_2}(s,a)$, as well as a pair of actors $\mu_{\theta_1}(s)$ and $\mu_{\theta_2}(s)$ (and all of their corresponding target networks).

Where $\mu_{\theta_1}(s)$ is optimized w.r.t. $Q_{w_1}(s,a)$ and $\mu_{\theta_2}(s)$ is optimized w.r.t. $Q_{w_2}(s,a)$.

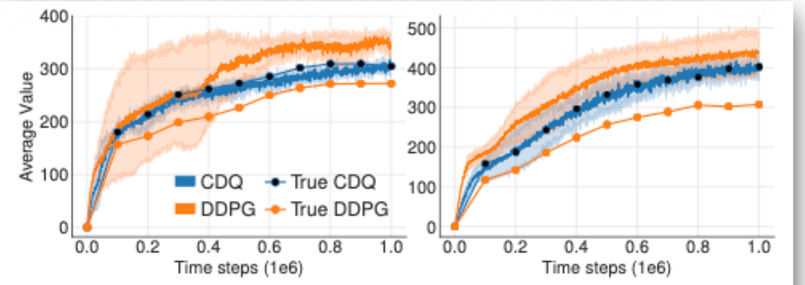
$$y_1 = r + \gamma \bar{Q}_{w_1}(s, \bar{\mu}_{\theta_2}(s))$$

$$y_2 = r + \gamma \bar{Q}_{w_2}(s, \bar{\mu}_{\theta_1}(s))$$

Clipped double Q-learning:

$$y = \min_{i=1,2} y_i \longrightarrow L(w_i) = \mathbb{E}\left[\left(Q_{w_i}(s,a) - y\right)^2\right]$$

With Clipped Double Q-learning, the value target cannot introduce any additional overestimation over using the standard Q-learning target. While this update rule may induce an underestimation bias, this is far preferable to overestimation bias, as unlike overestimated actions, the value of underestimated actions will not be explicitly propagated through the policy update.



(a) Hopper-v1 (b) Walker2d-v1

Figure 1. Measuring overestimation bias in the value estimates of DDPG and our proposed method, Clipped Double Q-learning (CDQ), on MuJoCo environments over 1 million time steps.

"The true value is estimated using the average discounted return over 1000 episodes following the current policy, starting from states sampled from the replay buffer."

Modification 2: Delayed policy updates

The problem: Updating the actor too frequently based on constantly shifting critic values can cause instability in the learning problem.

If target networks can be used to reduce the error over multiple updates, and policy updates on high-error states cause divergent behavior, then the policy network should be updated at a lower frequency than the value network, to first minimize error before introducing a policy update. We propose delaying policy updates until the value error is as small as possible. The modification is to only update the policy and target networks after a fixed number of updates d to the critic. To ensure the TD-error remains small, we update the target networks slowly $\theta' \leftarrow \tau\theta + (1-\tau)\theta'$.

The solution: Update the actor μ_θ (and target networks) less frequently than the critics Q_{w_1} and Q_{w_2}.

Intuition: Slow down the feedback loop caused by the dependence of μ_θ and Q_{w_1}, Q_{w_2} on each other's outputs.

Break the tight coupling between their learning dynamics, to prevent rapid propagation of errors between them.

Modification 3: Target policy smoothing

The problem: The actor can learn to exploit norrow, “brittle” peaks in the Q-function.

E.g., if $Q(s, a_{peak}) = 100$, but $Q(s, a_{peak} + \epsilon) = -100$, the actor may learn to take a_{peak}, even though any small errors could lead to drastically different values.

A concern with deterministic policies is they can overfit to narrow peaks in the value estimate. When updating the critic, a learning target using a deterministic policy is highly susceptible to inaccuracies induced by function approximation error, increasing the variance of the target. This induced variance can be reduced through regularization. We introduce a regularization strategy for deep value learning, target policy smoothing, which mimics the learning update from SARSA (Sutton & Barto, 1998). Our approach enforces the notion that similar actions should have similar value. While the function approximation does this implicitly, the relationship between similar actions can be forced explicitly by modifying the training procedure. We propose that fitting the value of a small area around the target action

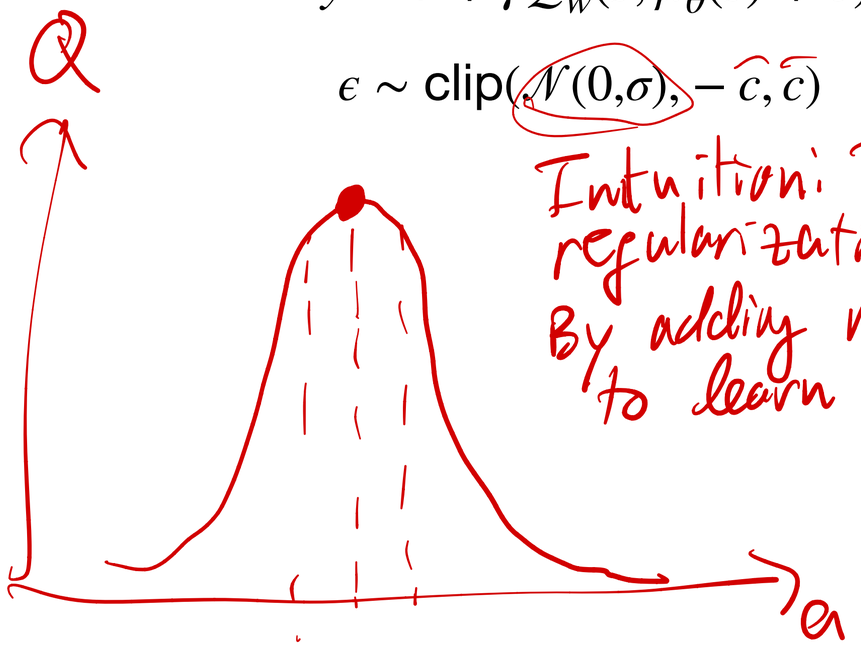
$$y = r + \mathbb{E}_\epsilon \left[Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon) \right], \tag{13}$$

would have the benefit of smoothing the value estimate by bootstrapping off of similar state-action value estimates. In

The solution: Add a small amount of clipped, random noise to the action used in the target y_i calculations.

$$y = r + \gamma \bar{Q}_w(s, \bar{\mu}_\theta(s) + \epsilon)$$

$$\epsilon \sim \text{clip}(\mathcal{N}(0,\sigma), -\widehat{c}, \widehat{c})$$



Intuition: This is a form of regularization on the critic learning. By adding noise, we force the critic to learn smoother values.

The TD3 algorithm

Algorithm 1 TD3

Initialize critic networks Q_{θ_1}, Q_{θ_2}, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ
Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$
Initialize replay buffer $\mathcal{B}$
for $t = 1$ **to** T **do**
  Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$, $\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'
  Store transition tuple (s, a, r, s') in $\mathcal{B}$

  Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$
  $\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$
  $y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$
  Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$
  if t mod d **then**
    Update ϕ by the deterministic policy gradient:
    $\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$
    Update target networks:
    $\theta'_i \leftarrow \tau\theta_i + (1 - \tau)\theta'_i$
    $\phi' \leftarrow \tau\phi + (1 - \tau)\phi'$
  end if
end for

Initialize actor, critic, and target

← sample exploration noise

← add to replay buffer

← sample minibatches

← sample clipped noise to "smooth" the critic's target

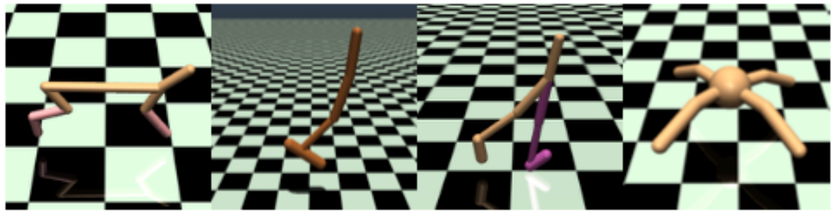
double Q-learning

Twin

delayed deterministic

once in a while, update actor network and the target networks.

TD3 Experimental results



(a) (b) (c) (d)

Figure 4. Example MuJoCo environments (a) HalfCheetah-v1, (b) Hopper-v1, (c) Walker2d-v1, (d) Ant-v1.

To evaluate our algorithm, we measure its performance on the suite of MuJoCo continuous control tasks (Todorov et al., 2012), interfaced through OpenAI Gym (Brockman et al., 2016) (Figure 4). To allow for reproducible comparison, we use the original set of tasks from Brockman et al. (2016) with no modifications to the environment or reward.

TD3 Experimental results

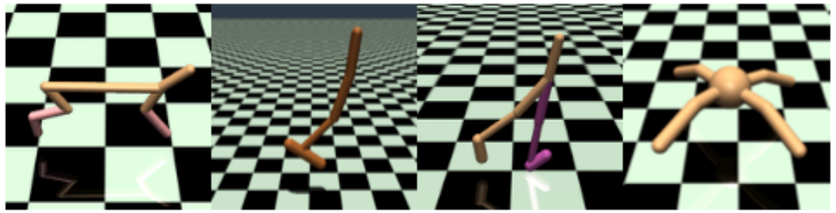


Figure 4. Example MuJoCo environments (a) HalfCheetah-v1, (b) Hopper-v1, (c) Walker2d-v1, (d) Ant-v1.

To evaluate our algorithm, we measure its performance on the suite of MuJoCo continuous control tasks (Todorov et al., 2012), interfaced through OpenAI Gym (Brockman et al., 2016) (Figure 4). To allow for reproducible comparison, we use the original set of tasks from Brockman et al. (2016) with no modifications to the environment or reward.

Our results are presented in Table 1 and learning curves in Figure 5. TD3 matches or outperforms all other algorithms in both final performance and learning speed across all tasks.

Taken together, these improvements define our proposed approach, the Twin Delayed Deep Deterministic policy gradient algorithm (TD3), which greatly improves both the learning speed and performance of DDPG in a number of challenging tasks in the continuous control setting. Our algorithm exceeds the performance of numerous state of the art algorithms. As our modifications are simple to implement, they can be easily added to any other actor-critic algorithm.

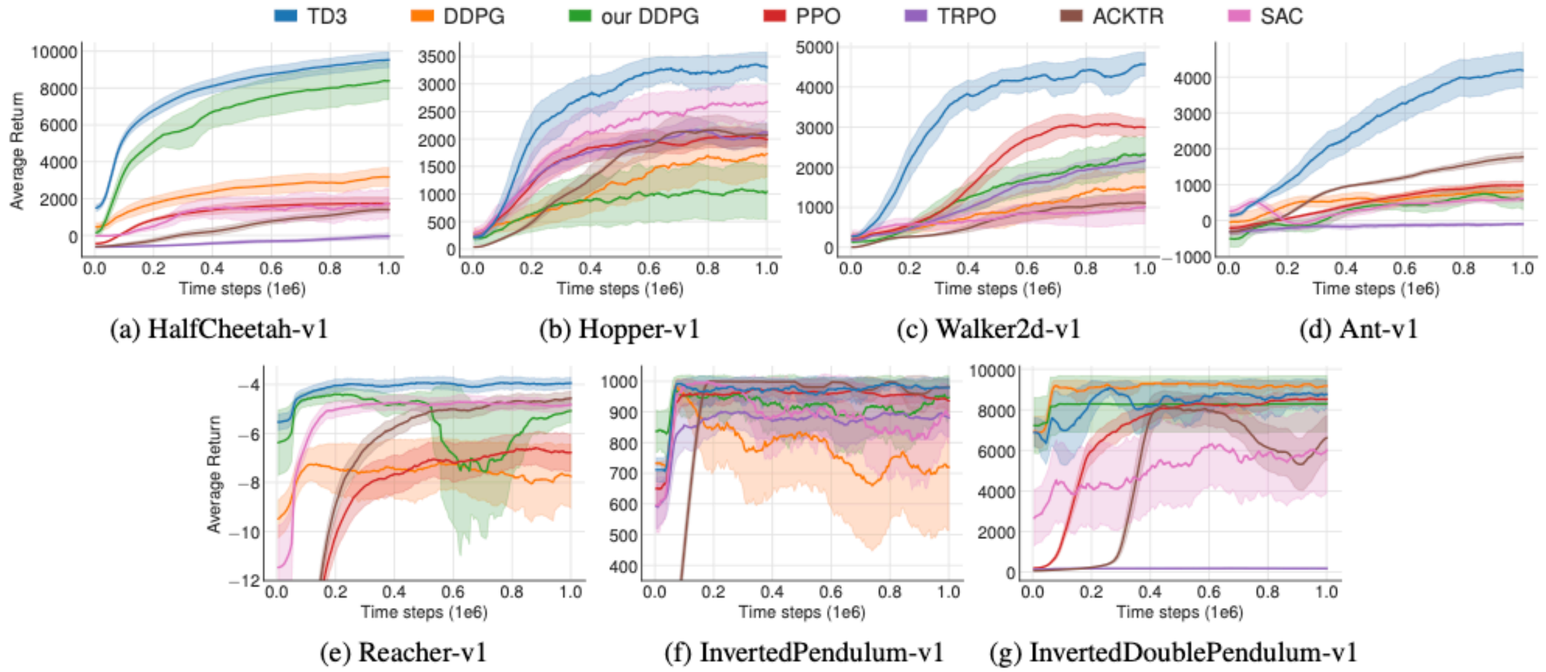


Figure 5. Learning curves for the OpenAI gym continuous control tasks. The shaded region represents half a standard deviation of the average evaluation over 10 trials. Curves are smoothed uniformly for visual clarity.

Summarizing DPG -> DDPG -> TD3

Off-policy, deterministic policy, actor-critic algorithms for continuous control.

1. Start by recognizing that $\max_a Q(s,a)$ is a bottleneck for applying RL to continuous control problems.
2. Used DPG theorem to replace $\max_a Q(s,a)$ in policy improvement step for parametrized actors.
3. Created DDPG by combining DPG theorem + AC architecture + DQN tricks (replay buffer, target networks) to make it work with neural networks.
4. Identified and fixed some of DDPG's flaws in TD3
   → Relatively robust and high-performing model free algorithm for continuous control.

So… did we solve RL yet?

No! We have a deterministic policy.

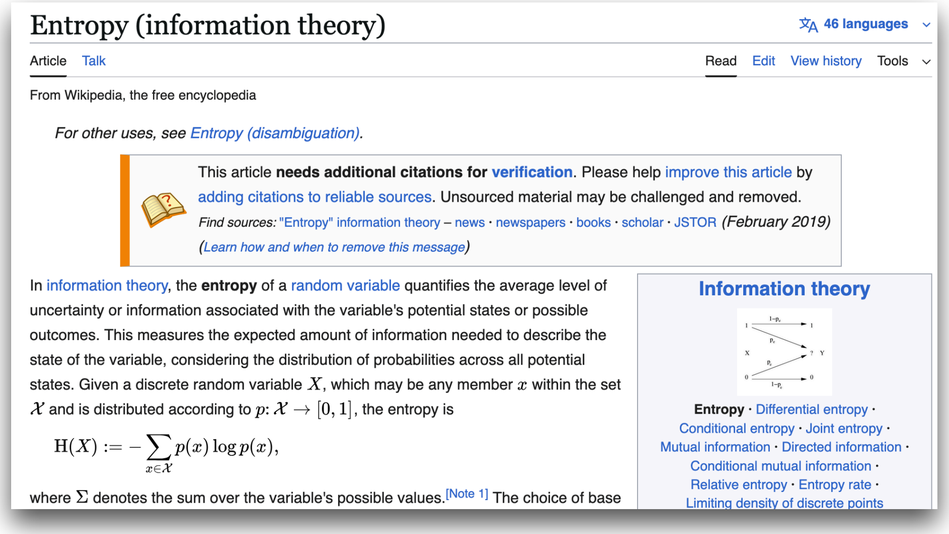
- Not great in game playing/adversarial setting.
- They can be brittle.
  - ↳ learns a single good action for each state. If the world is noisy, or our models are wrong, it's hard to recover if we get kicked out of states where the policy is performant.
- Multiple optimal solutions are possible: Deterministic policy chooses one.
- Exploration: Deterministic policies ⟶ don't explore. We add noise, but is "crude" exploration.

what can we do? maximize objective and maximize entropy (policy)

Entropy

Entropy is a formal measure of uncertainty, randomness, or unpredictability.

This concept is used in diverse fields: classical thermodynamics (where it was first developed), statistical physics, information theory, RL.



Entropy (information theory)

46 languages

Article Talk | Read Edit View history Tools

From Wikipedia, the free encyclopedia

For other uses, see Entropy (disambiguation).

This article **needs additional citations for verification**. Please help improve this article by adding citations to reliable sources. Unsourced material may be challenged and removed.

Find sources: "Entropy" information theory – news · newspapers · books · scholar · JSTOR *(February 2019)*

(Learn how and when to remove this message)

In information theory, the **entropy** of a random variable quantifies the average level of uncertainty or information associated with the variable's potential states or possible outcomes. This measures the expected amount of information needed to describe the state of the variable, considering the distribution of probabilities across all potential states. Given a discrete random variable X, which may be any member x within the set $\mathcal{X}$ and is distributed according to $p\colon \mathcal{X} \to [0, 1]$, the entropy is

$$\mathrm{H}(X) := -\sum_{x \in \mathcal{X}} p(x) \log p(x),$$

where Σ denotes the sum over the variable's possible values.[Note 1] The choice of base

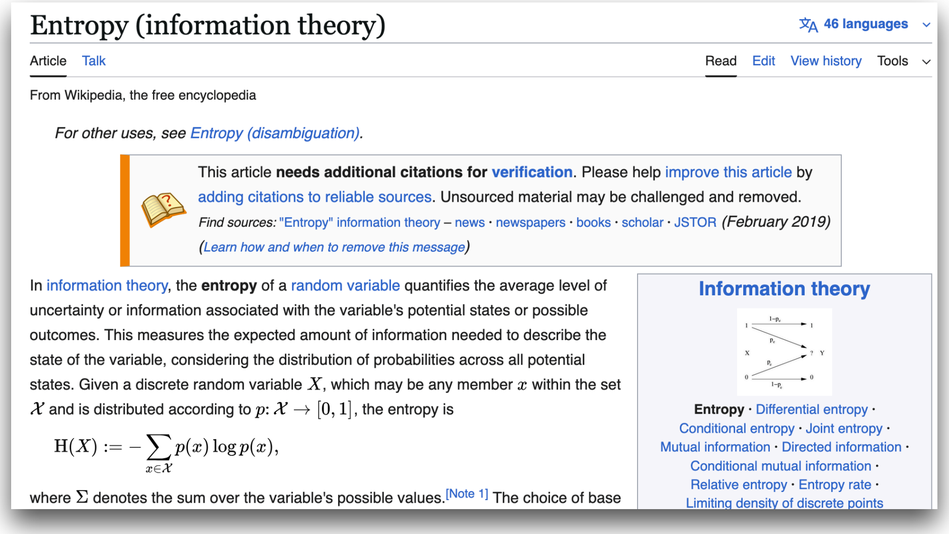
Information theory

Entropy · Differential entropy · Conditional entropy · Joint entropy · Mutual information · Directed information · Conditional mutual information · Relative entropy · Entropy rate · Limiting density of discrete points

Entropy

Entropy is a formal measure of uncertainty, randomness, or unpredictability.

This concept is used in diverse fields: classical thermodynamics (where it was first developed), statistical physics, information theory, RL.



Entropy (information theory)

46 languages

Article Talk | Read Edit View history Tools

From Wikipedia, the free encyclopedia

For other uses, see Entropy (disambiguation).

This article **needs additional citations for verification**. Please help improve this article by adding citations to reliable sources. Unsourced material may be challenged and removed.

Find sources: "Entropy" information theory – news · newspapers · books · scholar · JSTOR *(February 2019)*

(Learn how and when to remove this message)

In information theory, the **entropy** of a random variable quantifies the average level of uncertainty or information associated with the variable's potential states or possible outcomes. This measures the expected amount of information needed to describe the state of the variable, considering the distribution of probabilities across all potential states. Given a discrete random variable X, which may be any member x within the set $\mathcal{X}$ and is distributed according to $p\colon \mathcal{X} \to [0, 1]$, the entropy is

$$\mathrm{H}(X) := -\sum_{x \in \mathcal{X}} p(x) \log p(x),$$

where Σ denotes the sum over the variable's possible values.[Note 1] The choice of base

Information theory

Entropy · Differential entropy · Conditional entropy · Joint entropy · Mutual information · Directed information · Conditional mutual information · Relative entropy · Entropy rate · Limiting density of discrete points

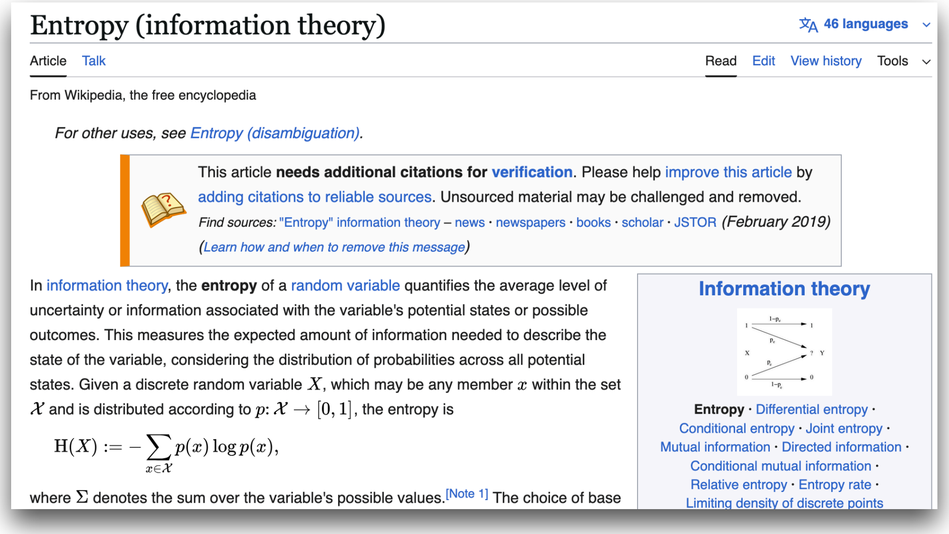
For discrete random variable X that can take on a set of possible values $\mathcal{X} = \{x_1, x_2, \ldots, x_n\}$,

$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

Entropy

Entropy is a formal measure of uncertainty, randomness, or unpredictability.

This concept is used in diverse fields: classical thermodynamics (where it was first developed), statistical physics, information theory, RL.



Entropy (information theory)

46 languages

Article Talk | Read Edit View history Tools

From Wikipedia, the free encyclopedia

For other uses, see Entropy (disambiguation).

This article **needs additional citations for verification**. Please help improve this article by adding citations to reliable sources. Unsourced material may be challenged and removed.

Find sources: "Entropy" information theory – news · newspapers · books · scholar · JSTOR *(February 2019)*

(Learn how and when to remove this message)

In information theory, the **entropy** of a random variable quantifies the average level of uncertainty or information associated with the variable's potential states or possible outcomes. This measures the expected amount of information needed to describe the state of the variable, considering the distribution of probabilities across all potential states. Given a discrete random variable X, which may be any member x within the set $\mathcal{X}$ and is distributed according to $p\colon \mathcal{X} \to [0, 1]$, the entropy is

$$\mathrm{H}(X) := -\sum_{x \in \mathcal{X}} p(x) \log p(x),$$

where Σ denotes the sum over the variable's possible values.[Note 1] The choice of base

Information theory

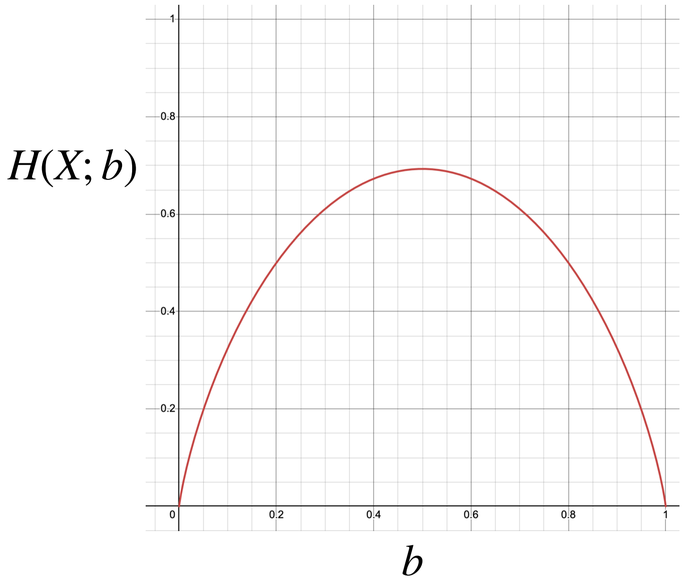
Entropy · Differential entropy · Conditional entropy · Joint entropy · Mutual information · Directed information · Conditional mutual information · Relative entropy · Entropy rate · Limiting density of discrete points

For discrete random variable X that can take on a set of possible values $\mathcal{X} = \{x_1, x_2, \ldots, x_n\}$,

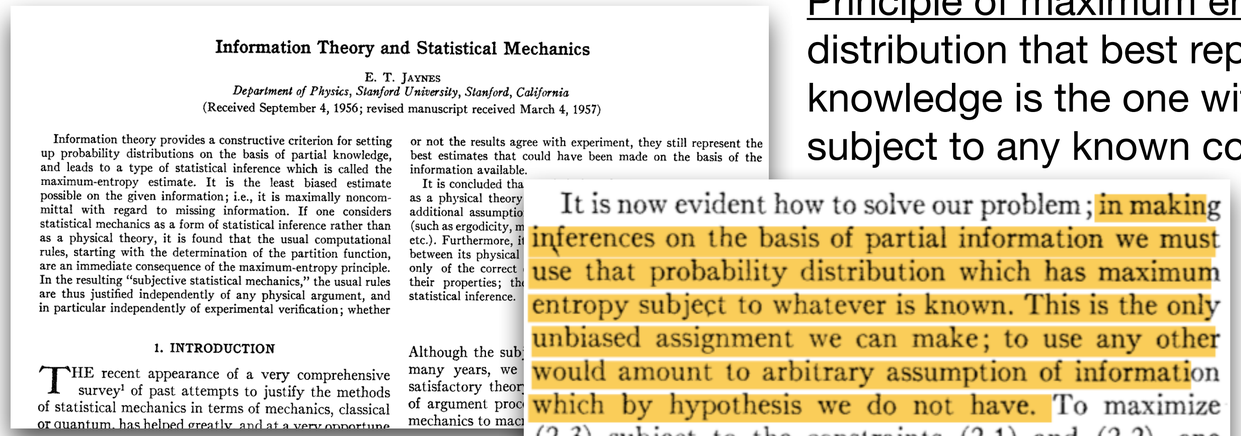
$$H(X) = -\sum_{x \in \mathcal{X}} p(x) \log p(x)$$

E.g., consider a Bernoulli r.v. X (biased coin flip) with parameter b.

$$H(X; b) = -b \log(b) - (1 - b) \log(b)$$



Maximum entropy ideas influencing RL



Information Theory and Statistical Mechanics

E. T. JAYNES

Department of Physics, Stanford University, Stanford, California

(Received September 4, 1956; revised manuscript received March 4, 1957)

Information theory provides a constructive criterion for setting up probability distributions on the basis of partial knowledge, and leads to a type of statistical inference which is called the maximum-entropy estimate. It is the least biased estimate possible on the given information; i.e., it is maximally noncommittal with regard to missing information. If one considers statistical mechanics as a form of statistical inference rather than as a physical theory, it is found that the usual computational rules, starting with the determination of the partition function, are an immediate consequence of the maximum-entropy principle. In the resulting "subjective statistical mechanics," the usual rules are thus justified independently of any physical argument, and in particular independently of experimental verification; whether or not the results agree with experiment, they still represent the best estimates that could have been made on the basis of the information available.

It is concluded tha
as a physical theory
additional assumptio
(such as ergodicity, m
etc.). Furthermore, it
between its physical
only of the correct
their properties; th
statistical inference.

1. INTRODUCTION

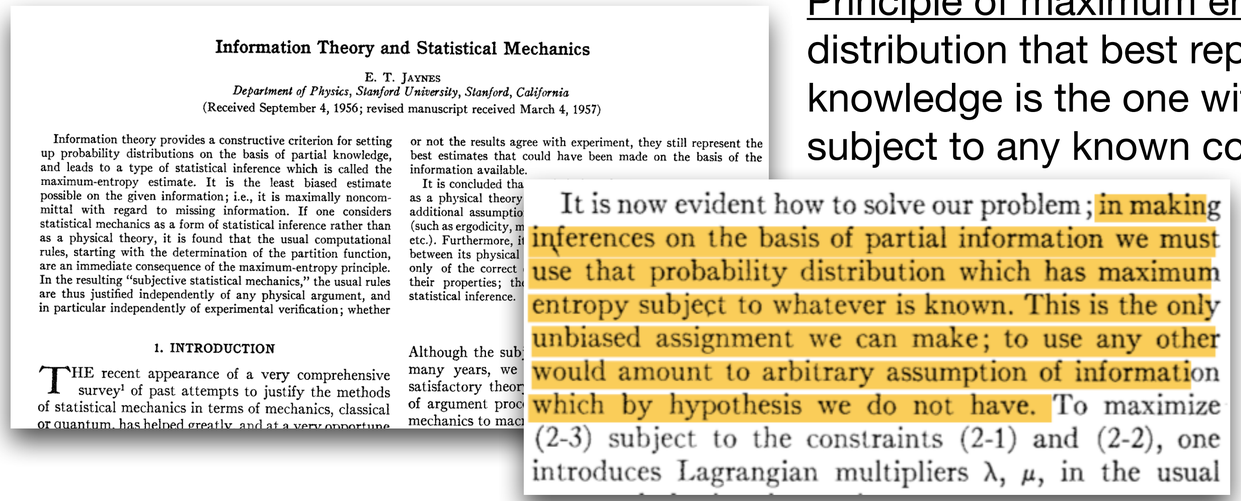
THE recent appearance of a very comprehensive survey[1] of past attempts to justify the methods of statistical mechanics in terms of mechanics, classical or quantum, has helped greatly, and at a very opportune

Although the subj
many years, we
satisfactory theor
of argument proc
mechanics to mac

It is now evident how to solve our problem; in making inferences on the basis of partial information we must use that probability distribution which has maximum entropy subject to whatever is known. This is the only unbiased assignment we can make; to use any other would amount to arbitrary assumption of information which by hypothesis we do not have. To maximize (2-3) subject to the constraints (2-1) and (2-2), one introduces Lagrangian multipliers λ, μ, in the usual

Principle of maximum entropy: the probability distribution that best represents the current state of knowledge is the one with the highest entropy, subject to any known constraints (e.g., data).

Maximum entropy ideas influencing RL



Information Theory and Statistical Mechanics

E. T. JAYNES
Department of Physics, Stanford University, Stanford, California
(Received September 4, 1956; revised manuscript received March 4, 1957)

Information theory provides a constructive criterion for setting up probability distributions on the basis of partial knowledge, and leads to a type of statistical inference which is called the maximum-entropy estimate. It is the least biased estimate possible on the given information; i.e., it is maximally noncommittal with regard to missing information. If one considers statistical mechanics as a form of statistical inference rather than as a physical theory, it is found that the usual computational rules, starting with the determination of the partition function, are an immediate consequence of the maximum-entropy principle. In the resulting "subjective statistical mechanics," the usual rules are thus justified independently of any physical argument, and in particular independently of experimental verification; whether or not the results agree with experiment, they still represent the best estimates that could have been made on the basis of the information available.

It is concluded tha
as a physical theory
additional assumptio
(such as ergodicity, m
etc.). Furthermore, it
between its physical
only of the correct
their properties; th
statistical inference.

1. INTRODUCTION

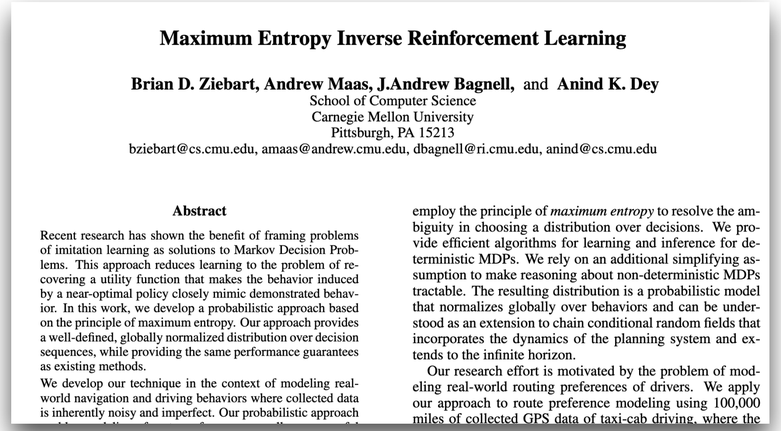
THE recent appearance of a very comprehensive survey[1] of past attempts to justify the methods of statistical mechanics in terms of mechanics, classical or quantum, has helped greatly, and at a very opportune

Although the subj
many years, we
satisfactory theor
of argument proc
mechanics to mac

It is now evident how to solve our problem; in making inferences on the basis of partial information we must use that probability distribution which has maximum entropy subject to whatever is known. This is the only unbiased assignment we can make; to use any other would amount to arbitrary assumption of information which by hypothesis we do not have. To maximize (2-3) subject to the constraints (2-1) and (2-2), one introduces Lagrangian multipliers λ, μ, in the usual

Principle of maximum entropy: the probability distribution that best represents the current state of knowledge is the one with the highest entropy, subject to any known constraints (e.g., data).

Apply principle of maximum entropy to IRL: Given a set of expert demonstrations, the agent is assumed to be acting to maximize rewards while also maximizing entropy. This gievs a robust way to handle ambiguity in estimating the reward function from demonstrations.



Maximum Entropy Inverse Reinforcement Learning

Brian D. Ziebart, Andrew Maas, J.Andrew Bagnell, and **Anind K. Dey**
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213
bziebart@cs.cmu.edu, amaas@andrew.cmu.edu, dbagnell@ri.cmu.edu, anind@cs.cmu.edu

Abstract

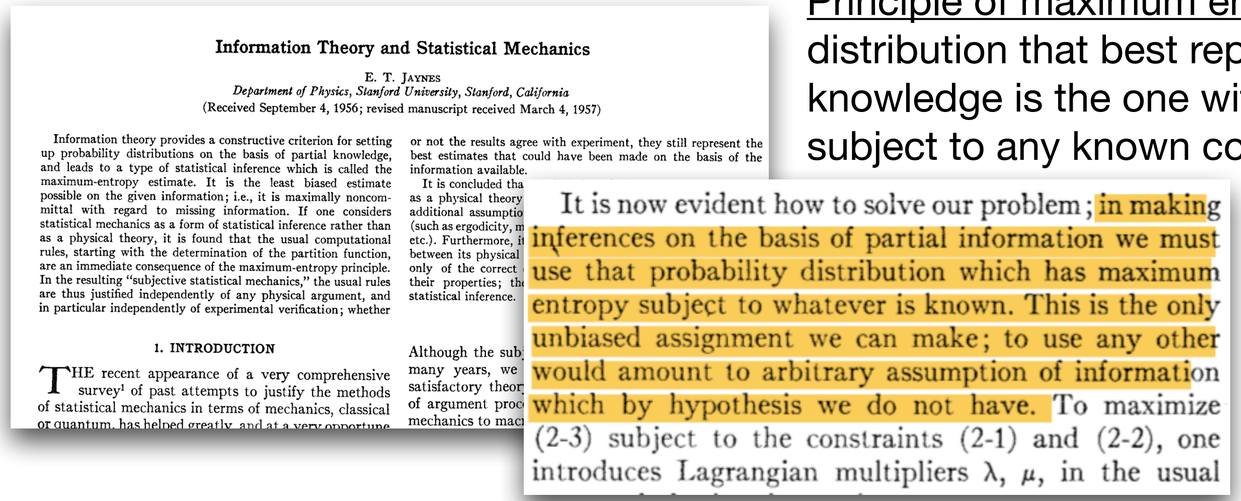
Recent research has shown the benefit of framing problems of imitation learning as solutions to Markov Decision Problems. This approach reduces learning to the problem of recovering a utility function that makes the behavior induced by a near-optimal policy closely mimic demonstrated behavior. In this work, we develop a probabilistic approach based on the principle of maximum entropy. Our approach provides a well-defined, globally normalized distribution over decision sequences, while providing the same performance guarantees as existing methods.

We develop our technique in the context of modeling real-world navigation and driving behaviors where collected data is inherently noisy and imperfect. Our probabilistic approach

employ the principle of *maximum entropy* to resolve the ambiguity in choosing a distribution over decisions. We provide efficient algorithms for learning and inference for deterministic MDPs. We rely on an additional simplifying assumption to make reasoning about non-deterministic MDPs tractable. The resulting distribution is a probabilistic model that normalizes globally over behaviors and can be understood as an extension to chain conditional random fields that incorporates the dynamics of the planning system and extends to the infinite horizon.

Our research effort is motivated by the problem of modeling real-world routing preferences of drivers. We apply our approach to route preference modeling using 100,000 miles of collected GPS data of taxi-cab driving, where the

Maximum entropy ideas influencing RL



Information Theory and Statistical Mechanics

E. T. Jaynes
Department of Physics, Stanford University, Stanford, California
(Received September 4, 1956; revised manuscript received March 4, 1957)

Information theory provides a constructive criterion for setting up probability distributions on the basis of partial knowledge, and leads to a type of statistical inference which is called the maximum-entropy estimate. It is the least biased estimate possible on the given information; i.e., it is maximally noncommittal with regard to missing information. If one considers statistical mechanics as a form of statistical inference rather than as a physical theory, it is found that the usual computational rules, starting with the determination of the partition function, are an immediate consequence of the maximum-entropy principle. In the resulting "subjective statistical mechanics," the usual rules are thus justified independently of any physical argument, and in particular independently of experimental verification; whether or not the results agree with experiment, they still represent the best estimates that could have been made on the basis of the information available.

It is concluded tha
as a physical theory
additional assumptio
(such as ergodicity, m
etc.). Furthermore, it
between its physical
only of the correct
their properties; th
statistical inference.

1. INTRODUCTION

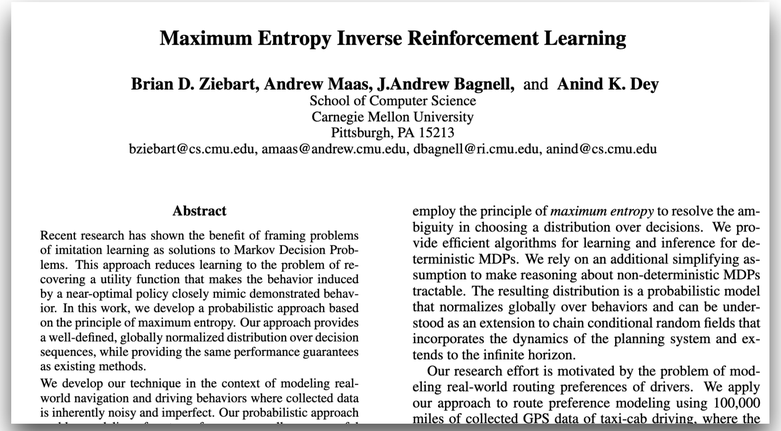
THE recent appearance of a very comprehensive survey[1] of past attempts to justify the methods of statistical mechanics in terms of mechanics, classical or quantum, has helped greatly, and at a very opportune

Although the subj
many years, we
satisfactory theor
of argument proc
mechanics to mac

It is now evident how to solve our problem; in making inferences on the basis of partial information we must use that probability distribution which has maximum entropy subject to whatever is known. This is the only unbiased assignment we can make; to use any other would amount to arbitrary assumption of information which by hypothesis we do not have. To maximize (2-3) subject to the constraints (2-1) and (2-2), one introduces Lagrangian multipliers λ, μ, in the usual

Principle of maximum entropy: the probability distribution that best represents the current state of knowledge is the one with the highest entropy, subject to any known constraints (e.g., data).

Apply principle of maximum entropy to IRL: Given a set of expert demonstrations, the agent is assumed to be acting to maximize rewards while also maximizing entropy. This gievs a robust way to handle ambiguity in estimating the reward function from demonstrations.



Maximum Entropy Inverse Reinforcement Learning

Brian D. Ziebart, Andrew Maas, J.Andrew Bagnell, and **Anind K. Dey**
School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213
bziebart@cs.cmu.edu, amaas@andrew.cmu.edu, dbagnell@ri.cmu.edu, anind@cs.cmu.edu

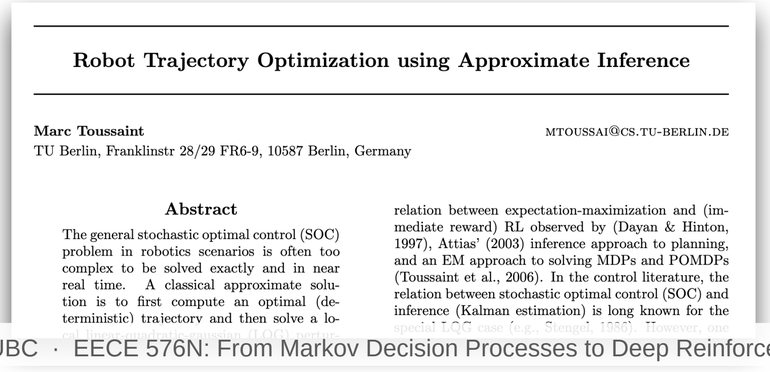
Abstract

Recent research has shown the benefit of framing problems of imitation learning as solutions to Markov Decision Problems. This approach reduces learning to the problem of recovering a utility function that makes the behavior induced by a near-optimal policy closely mimic demonstrated behavior. In this work, we develop a probabilistic approach based on the principle of maximum entropy. Our approach provides a well-defined, globally normalized distribution over decision sequences, while providing the same performance guarantees as existing methods.

We develop our technique in the context of modeling real-world navigation and driving behaviors where collected data is inherently noisy and imperfect. Our probabilistic approach

employ the principle of *maximum entropy* to resolve the ambiguity in choosing a distribution over decisions. We provide efficient algorithms for learning and inference for deterministic MDPs. We rely on an additional simplifying assumption to make reasoning about non-deterministic MDPs tractable. The resulting distribution is a probabilistic model that normalizes globally over behaviors and can be understood as an extension to chain conditional random fields that incorporates the dynamics of the planning system and extends to the infinite horizon.

Our research effort is motivated by the problem of modeling real-world routing preferences of drivers. We apply our approach to route preference modeling using 100,000 miles of collected GPS data of taxi-cab driving, where the



Robot Trajectory Optimization using Approximate Inference

Marc Toussaint MTOUSSAI@CS.TU-BERLIN.DE
TU Berlin, Franklinstr 28/29 FR6-9, 10587 Berlin, Germany

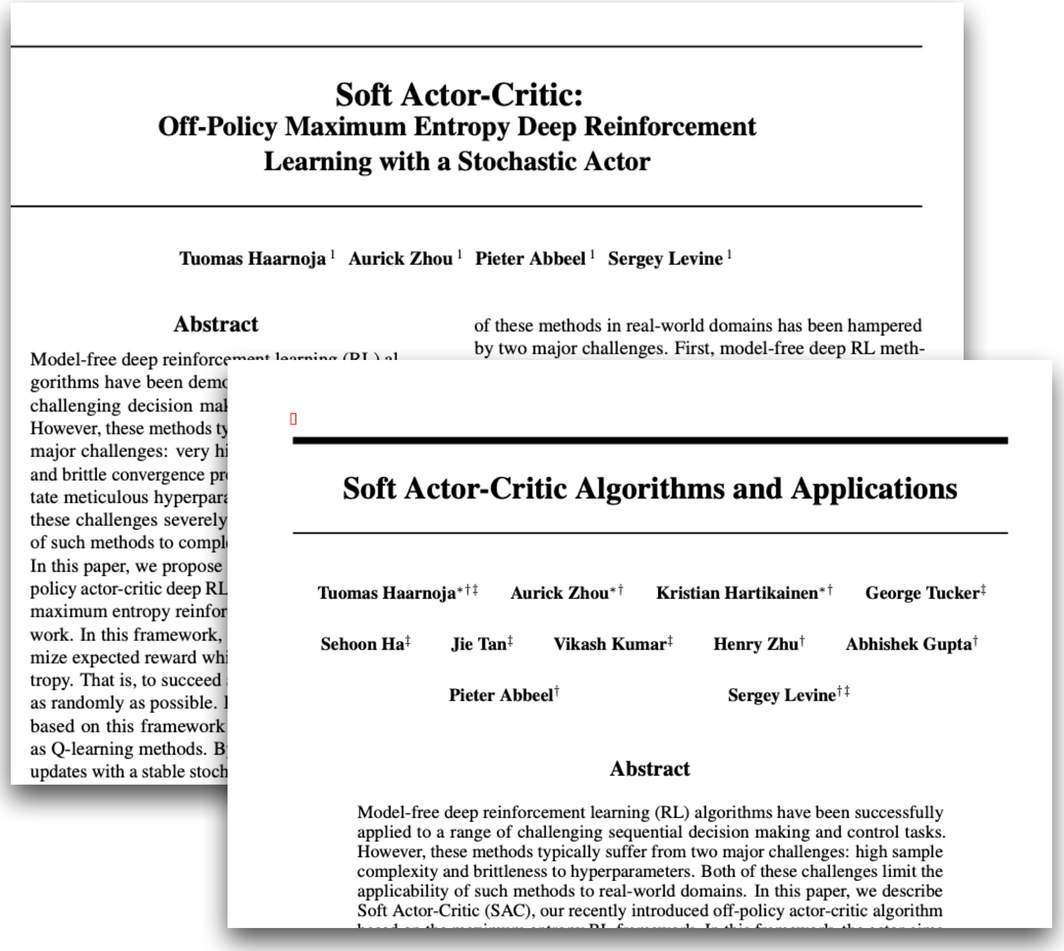
Abstract

The general stochastic optimal control (SOC) problem in robotics scenarios is often too complex to be solved exactly and in near real time. A classical approximate solution is to first compute an optimal (deterministic) trajectory and then solve a lo-

relation between expectation-maximization and (immediate reward) RL observed by (Dayan & Hinton, 1997), Attias' (2003) inference approach to planning, and an EM approach to solving MDPs and POMDPs (Toussaint et al., 2006). In the control literature, the relation between stochastic optimal control (SOC) and inference (Kalman estimation) is long known for the special LQG case (e.g., Stengel, 1986). However, one

Control as inference: Reformulate trajectory optimization as a probabilistic inference problem, a core concept that defines max-entropy objectives later operationalized by algorithms like SAC.

Soft Actor-Critic



Soft Actor-Critic:
Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor

Tuomas Haarnoja [1] Aurick Zhou [1] Pieter Abbeel [1] Sergey Levine [1]

Abstract

Model-free deep reinforcement learning (RL) al-
gorithms have been demo
challenging decision mak
However, these methods ty
major challenges: very hi
and brittle convergence pr
tate meticulous hyperpara
these challenges severely
of such methods to compl
In this paper, we propose
policy actor-critic deep RL
maximum entropy reinfor
work. In this framework,
mize expected reward whi
tropy. That is, to succeed
as randomly as possible.
based on this framework
as Q-learning methods. B
updates with a stable stoch

of these methods in real-world domains has been hampered by two major challenges. First, model-free deep RL meth-

Soft Actor-Critic Algorithms and Applications

Tuomas Haarnoja*†‡ Aurick Zhou*† Kristian Hartikainen*† George Tucker‡
Sehoon Ha‡ Jie Tan‡ Vikash Kumar‡ Henry Zhu† Abhishek Gupta†
Pieter Abbeel† Sergey Levine†‡

Abstract

Model-free deep reinforcement learning (RL) algorithms have been successfully applied to a range of challenging sequential decision making and control tasks. However, these methods typically suffer from two major challenges: high sample complexity and brittleness to hyperparameters. Both of these challenges limit the applicability of such methods to real-world domains. In this paper, we describe Soft Actor-Critic (SAC), our recently introduced off-policy actor-critic algorithm

Key idea: Maximum entropy version of DDPG algorithm.

Maximum entropy RL objective:

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[r(s_t, a_t)\right] \qquad \text{standard}$$

$$\downarrow$$

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[r(s_t, a_t) + \alpha H(\pi(\cdot \mid s_t))\right] \qquad \text{Maximum entropy}$$

where $H(\pi(\cdot \mid s_t)) = \mathbb{E}_{a\sim\pi(\cdot|s_t)}\left[-\log \pi(a \mid s_t)\right]$

Note: if $\alpha \to 0$, we recover standard RL.
if $\alpha \to \infty$, reward doesn't matter, becomes pure exploration problem.

Why learn maximum entropy policies?

Max-ent objective asks the algorithm to "find the policy that succeeds at the task while acting as randomly as possible".

- Exploration: Training the actor to get as much information out of it's environment as possible.
- Robustness: Make as few assumptions as possible when learning policy.
  ↳ learns a distribution of good actions, in case any such "assumption" was wrong.
- Multi-modal solutions: Capture all good solutions to the problem.
  ↳ later can fine-tune to more specific behaviors as needed.

The soft Bellman equation

The max-entropy objective $J(\pi)$ changes the standard Bellman equations.

$$J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[r(s_t,a_t) + \alpha H(\pi(\cdot \mid s_t))\right]$$

The soft Bellman equation

The max-entropy objective $J(\pi)$ changes the standard Bellman equations. $J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[r(s_t,a_t) + \alpha H(\pi(\cdot \mid s_t))\right]$

Introduce the *soft* value functions: $V_{soft}(s) = \mathbb{E}_{a\sim\pi}\left[Q_{soft}(s,a) - \alpha \log \pi(a \mid s)\right]$ $Q_{soft}(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[V_{soft}(s')\right]$

And the modified bellman backup operator $\mathcal{T}^\pi Q(s,a) = r(s,a) + \gamma \mathcal{E}_{s'}\left[V_{soft}(s')\right]$

The soft Bellman equation

The max-entropy objective $J(\pi)$ changes the standard Bellman equations. $J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[r(s_t,a_t) + \alpha H(\pi(\cdot \mid s_t))\right]$

Introduce the *soft* value functions: $V_{soft}(s) = \mathbb{E}_{a\sim\pi}\left[Q_{soft}(s,a) - \alpha \log \pi(a \mid s)\right]$ $Q_{soft}(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[V_{soft}(s')\right]$

And the modified bellman backup operator $\mathcal{T}^\pi Q(s,a) = r(s,a) + \gamma \mathcal{E}_{s'}\left[V_{soft}(s')\right]$

Optimal policy π^*:

Suppose $Q^*_{soft}(s,a)$ is the optimal soft Q function.

In standard RL, the optimal action is given by a maximization

$$a^* \in \arg\max_{a\in A} Q(s,a)$$

The soft Bellman equation

The max-entropy objective $J(\pi)$ changes the standard Bellman equations. $J(\pi) = \sum_t \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[r(s_t,a_t) + \alpha H(\pi(\cdot \mid s_t))\right]$

Introduce the *soft* value functions: $V_{soft}(s) = \mathbb{E}_{a\sim\pi}\left[Q_{soft}(s,a) - \alpha \log \pi(a \mid s)\right]$ $Q_{soft}(s,a) = r(s,a) + \gamma \mathbb{E}_{s'}\left[V_{soft}(s')\right]$

And the modified bellman backup operator $\mathcal{T}^\pi Q(s,a) = r(s,a) + \gamma \mathcal{E}_{s'}\left[V_{soft}(s')\right]$

Optimal policy π^*:

Suppose $Q^*_{soft}(s,a)$ is the optimal soft Q function.

In standard RL, the optimal action is given by a maximization

$$a^* \in \arg\max_{a\in A} Q(s,a)$$

In max-entropy RL, the optimal policy is given by a softmax

$$\pi^*(a \mid s) = \frac{\exp\left(\frac{1}{\alpha} Q^*_{soft}(s,a)\right)}{Z(s)}$$

Where $Z(s)$ is some normalization constant.

Generalized policy iteration in max-entropy RL

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-value of π as $k \rightarrow \infty$.*

$$\mathcal{T}^\pi Q(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a' \sim \rho^\pi} \left[Q_{soft}(s,a) - \alpha \log \pi(a' \mid s') \right]$$

Iterate: $Q^{k+1} = \mathcal{T}^\pi Q^k$

Generalized policy iteration in max-entropy RL

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-value of π as $k \rightarrow \infty$.*

$$\mathcal{T}^\pi Q(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a' \sim \rho^\pi} \left[Q_{soft}(s,a) - \alpha \log \pi(a' \mid s') \right]$$

Iterate: $Q^{k+1} = \mathcal{T}^\pi Q^k$

Lemma 2 (Soft Policy Improvement). *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

$$\pi_{new} = \arg\min_{\pi' \in \Pi} D_{KL} \left(\pi'(\cdot \mid s_t) \,||\, \frac{\exp\left(Q^{\pi_{old}}(s, \cdot)\right)}{Z^{\pi_{old}}(s)} \right)$$

Generalized policy iteration in max-entropy RL

Lemma 1 (Soft Policy Evaluation). *Consider the soft Bellman backup operator $\mathcal{T}^\pi$ in Equation 2 and a mapping $Q^0 : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ with $|\mathcal{A}| < \infty$, and define $Q^{k+1} = \mathcal{T}^\pi Q^k$. Then the sequence Q^k will converge to the soft Q-value of π as $k \rightarrow \infty$.*

Lemma 2 (Soft Policy Improvement). *Let $\pi_{\text{old}} \in \Pi$ and let π_{new} be the optimizer of the minimization problem defined in Equation 4. Then $Q^{\pi_{\text{new}}}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^{\pi_{\text{old}}}(\mathbf{s}_t, \mathbf{a}_t)$ for all $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$ with $|\mathcal{A}| < \infty$.*

$$\mathcal{T}^\pi Q(s,a) = r(s,a) + \gamma \mathbb{E}_{s',a' \sim \rho^\pi} \left[Q_{soft}(s,a) - \alpha \log \pi(a' \mid s') \right]$$

$$\pi_{new} = \arg\min_{\pi' \in \Pi} D_{KL} \left(\pi'(\cdot \mid s_t) \,||\, \frac{\exp\left(Q^{\pi_{old}}(s, \cdot)\right)}{Z^{\pi_{old}}(s)} \right)$$

Iterate: $Q^{k+1} = \mathcal{T}^\pi Q^k$

Theorem 1 (Soft Policy Iteration). *Repeated application of soft policy evaluation and soft policy improvement from any $\pi \in \Pi$ converges to a policy π^* such that $Q^{\pi^*}(\mathbf{s}_t, \mathbf{a}_t) \geq Q^\pi(\mathbf{s}_t, \mathbf{a}_t)$ for all $\pi \in \Pi$ and $(\mathbf{s}_t, \mathbf{a}_t) \in \mathcal{S} \times \mathcal{A}$, assuming $|\mathcal{A}| < \infty$.*

The soft actor-critic (SAC) algorithm

SAC is an off-policy, actor-critic algorithm that implements this max-entropy RL framework. It learns:

1. A stochastic actor $\pi_\theta(a \mid s)$ (e.g., a Gaussian policy whose mean and variance are output by neural networks.
2. Two “soft” critic networks $Q_{w_1}(s, a)$ and $Q_{w_2}(s, a)$.
3. Eventually we’ll see, that the algorithm can also automatically learn hyperparameter α.

Note, the algorithm uses an experience replay buffer (off-policy) and twin critics (like TD3) for stability.

Critic update

Idea: train the Q-functions to minimize the soft-Bellman residual:

$$J_Q(w) = \mathbb{E}_{(s,a)\sim D}\left[\frac{1}{2}\left(Q_w(s,a) - \left(r(s,a) + \gamma \mathbb{E}_{s',a'}\left[\bar{Q}_w(s',a') - \alpha \log(\pi_\theta(a' \mid s'))\right]\right)\right)^2\right]$$

Critic update

Idea: train the Q-functions to minimize the soft-Bellman residual:

$$J_Q(w) = \mathbb{E}_{(s,a)\sim D}\left[\frac{1}{2}\left(Q_w(s,a) - \left(r(s,a) + \gamma \mathbb{E}_{s',a'}\left[\bar{Q}_w(s',a') - \alpha \log(\pi_\theta(a' \mid s'))\right]\right)\right)^2\right]$$

Practically: For each minibatch sample $(s_i, a_i, r_i, s_i') \sim D$:

1. Calculate the soft target y_i
    a. Get the next action and its log-probability from current (online) policy:

$$a_i', \log \pi_\theta(a_i' \mid s_i') \sim \pi_\theta(\cdot \mid s_i')$$

Critic update

Idea: train the Q-functions to minimize the soft-Bellman residual:

$$J_Q(w) = \mathbb{E}_{(s,a)\sim D}\left[\frac{1}{2}\left(Q_w(s,a) - \left(r(s,a) + \gamma \mathbb{E}_{s',a'}\left[\bar{Q}_w(s',a') - \alpha \log(\pi_\theta(a' \mid s'))\right]\right)\right)^2\right]$$

Practically: For each minibatch sample $(s_i, a_i, r_i, s_i') \sim D$:

1. Calculate the soft target y_i
    a. Get the next action and its log-probability from current (online) policy:

$$a_i', \log \pi_\theta(a_i' \mid s_i') \sim \pi_\theta(\cdot \mid s_i')$$

    b. Get the next Q-values from the **Target critics** (like in TD3):

$$Q_1' = \bar{Q}_{w_1}(s_i', a_i') \qquad Q_2' = \bar{Q}_{w_2}(s_i', a_i')$$

Soft target Q-function parameters $\bar{w}$ are obtained as an exponentially moving average of the soft Q-function weights w.

Critic update

Idea: train the Q-functions to minimize the soft-Bellman residual:

$$J_Q(w) = \mathbb{E}_{(s,a)\sim D}\left[\frac{1}{2}\left(Q_w(s,a) - \left(r(s,a) + \gamma \mathbb{E}_{s',a'}\left[\bar{Q}_w(s',a') - \alpha \log(\pi_\theta(a' \mid s'))\right]\right)\right)^2\right]$$

Practically: For each minibatch sample $(s_i, a_i, r_i, s_i') \sim D$:

1. Calculate the soft target y_i

   a. Get the next action and its log-probability from current (online) policy:

   $$a_i', \log \pi_\theta(a_i' \mid s_i') \sim \pi_\theta(\cdot \mid s_i')$$

   b. Get the next Q-values from the **Target critics** (like in TD3):

   $$Q_1' = \bar{Q}_{w_1}(s_i', a_i') \qquad Q_2' = \bar{Q}_{w_2}(s_i', a_i')$$

   c. Form the soft TD target:

   $$y_i = r_i + \gamma\left(\min(Q_1', Q_2') - \alpha \log \pi_\theta(a_i' \mid s_i')\right)$$

Soft target Q-function parameters $\bar{w}$ are obtained as an exponentially moving average of the soft Q-function weights w.

Critic update

Idea: train the Q-functions to minimize the soft-Bellman residual:

$$J_Q(w) = \mathbb{E}_{(s,a)\sim D}\left[\frac{1}{2}\left(Q_w(s,a) - \left(r(s,a) + \gamma \mathbb{E}_{s',a'}\left[\bar{Q}_w(s',a') - \alpha \log(\pi_\theta(a' \mid s'))\right]\right)\right)^2\right]$$

Practically: For each minibatch sample $(s_i, a_i, r_i, s_i') \sim D$:

1. Calculate the soft target y_i

   a. Get the next action and its log-probability from current (online) policy:

$$a_i', \log \pi_\theta(a_i' \mid s_i') \sim \pi_\theta(\cdot \mid s_i')$$

   b. Get the next Q-values from the **Target critics** (like in TD3):

$$Q_1' = \bar{Q}_{w_1}(s_i', a_i') \qquad Q_2' = \bar{Q}_{w_2}(s_i', a_i')$$

   Soft target Q-function parameters $\bar{w}$ are obtained as an exponentially moving average of the soft Q-function weights w.

   c. Form the soft TD target:

$$y_i = r_i + \gamma\left(\min(Q_1', Q_2') - \alpha \log \pi_\theta(a_i' \mid s_i')\right)$$

2. Update the critics by performing SGD on the loss:

$$L(w_1, w_2) = \frac{1}{N}\sum_i \left(Q_{w_1}(s_i, a_i) - y_i\right)^2 + \left(Q_{w_2}(s_i, a_i) - y_i\right)^2$$

Actor update

Idea: the actor $\pi_\theta(a \mid s)$ wants to maximize the soft value. It's objective function is:

$$J_\pi(\theta) = \mathbb{E}_{s\sim D}\left[\mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_w(s,a)\right]\right]$$

Find actions that have high Q-value and a high entropy (low log-prob).

Actor update

Idea: the actor $\pi_\theta(a \mid s)$ wants to maximize the soft value. It's objective function is:

$$J_\pi(\theta) = \mathbb{E}_{s\sim D}\left[\mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_w(s,a)\right]\right]$$

Find actions that have high Q-value and a high entropy (low log-prob).

Problem: How do we take a gradient ∇_θ of an objective that *samples* $a \sim \pi_\theta$? The "sample" operation is a stochastic, non-differentiable operation. We can't backpropagate through it.

Actor update

Idea: the actor $\pi_\theta(a \mid s)$ wants to maximize the soft value. It's objective function is:

$$J_\pi(\theta) = \mathbb{E}_{s\sim D}\left[\mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_w(s,a)\right]\right]$$

Find actions that have high Q-value and a high entropy (low log-prob).

Problem: How do we take a gradient ∇_θ of an objective that *samples* $a \sim \pi_\theta$? The "sample" operation is a stochastic, non-differentiable operation. We can't backpropagate through it.

The solution: The Reparameterization trick.

> An idea brought over from work in variational autoencoders: Make the sampling process differentiable by separating the network's deterministic outputs from random noise.

Actor update

Idea: the actor $\pi_\theta(a \mid s)$ wants to maximize the soft value. It's objective function is:

$$J_\pi(\theta) = \mathbb{E}_{s\sim D}\left[\mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_w(s,a)\right]\right]$$

Find actions that have high Q-value and a high entropy (low log-prob).

Problem: How do we take a gradient ∇_θ of an objective that *samples* $a \sim \pi_\theta$? The "sample" operation is a stochastic, non-differentiable operation. We can't backpropagate through it.

The solution: The Reparameterization trick.

An idea brought over from work in variational autoencoders: Make the sampling process differentiable by separating the network's deterministic outputs from random noise.

Instead of saying the actor samples $a \sim \pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$, we write:

$$a(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \cdot \epsilon$$

Where $\epsilon \sim \mathcal{N}(0,1)$ is a fixed noise source, external to the network.

Actor update

Idea: the actor $\pi_\theta(a \mid s)$ wants to maximize the soft value. It's objective function is:

$$J_\pi(\theta) = \mathbb{E}_{s\sim D}\left[\mathbb{E}_{a\sim\pi_\theta}\left[\alpha \log \pi_\theta(a \mid s) - Q_w(s,a)\right]\right]$$

Find actions that have high Q-value and a high entropy (low log-prob).

Problem: How do we take a gradient ∇_θ of an objective that *samples* $a \sim \pi_\theta$? The "sample" operation is a stochastic, non-differentiable operation. We can't backpropagate through it.

The solution: The Reparameterization trick.

An idea brought over from work in variational autoencoders: Make the sampling process differentiable by separating the network's deterministic outputs from random noise.

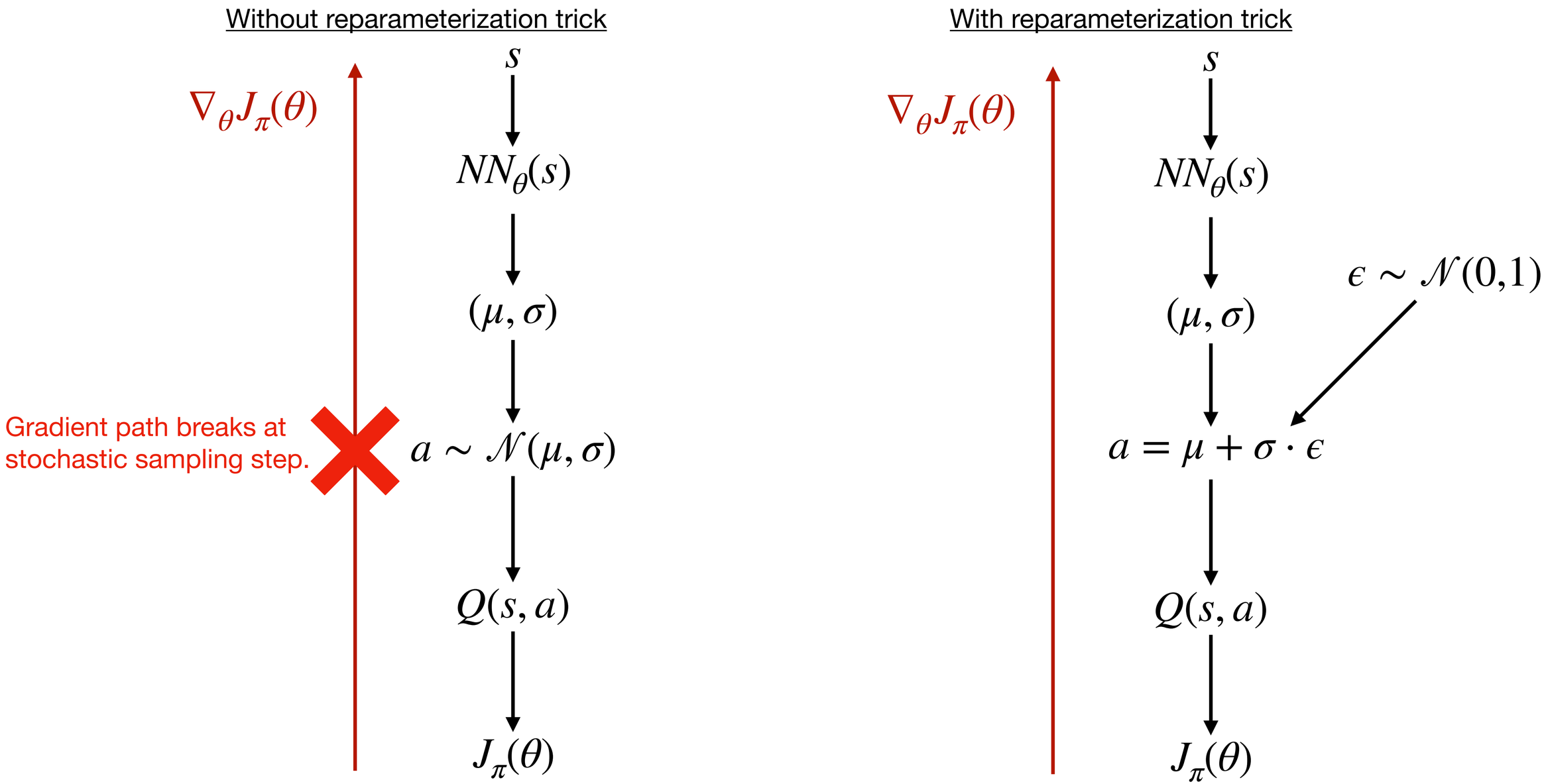
Instead of saying the actor samples $a \sim \pi_\theta(a \mid s) = \mathcal{N}(\mu_\theta(s), \sigma_\theta(s))$, we write:

$$a(s,\epsilon) = \mu_\theta(s) + \sigma_\theta(s) \cdot \epsilon$$

Where $\epsilon \sim \mathcal{N}(0,1)$ is a fixed noise source, external to the network.

This trick allows the actor network to just output parameters $(\mu_\theta, \sigma_\theta)$ of the policy. The stochasticity comes form an external, fixed source. The gradient $\nabla_\theta J_\pi(\theta)$ can flow back through the deterministic calculation $a = \mu + \sigma \cdot \epsilon$, through the Q-function, and update the actor's weights.

The reparameterization trick



Removing hyperparameters: simultaneously learning α

Problem: Algorithm performance is sensitive to hyperparameter α.

If α is too high, the agent acts randomly and never solves the task.

If α is too low, the policy will be deterministic, and we loose the benefits of max-entropy RL.

Removing hyperparameters: simultaneously learning α

Problem: Algorithm performance is sensitive to hyperparameter α.

If α is too high, the agent acts randomly and never solves the task.

If α is too low, the policy will be deterministic, and we loose the benefits of max-entropy RL.

Solution: Learn α by formulating a constrained optimization problem: force the policy's average entropy to be equal to some target entropy $\bar{H}$.

$$\max_{\pi_{0:T}} \mathbb{E}_{\rho^\pi}\left[\sum_{t=0}^{T} r(s_t, a_t)\right] \quad s.t. \quad \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[-\log \pi_t(a_t \mid s_t)\right] \geq H \,\forall t$$

Removing hyperparameters: simultaneously learning α

Problem: Algorithm performance is sensitive to hyperparameter α.

If α is too high, the agent acts randomly and never solves the task.

If α is too low, the policy will be deterministic, and we loose the benefits of max-entropy RL.

Solution: Learn α by formulating a constrained optimization problem: force the policy's average entropy to be equal to some target entropy $\bar{H}$.

$$\max_{\pi_{0:T}} \mathbb{E}_{\rho^\pi}\left[\sum_{t=0}^{T} r(s_t, a_t)\right] \quad s.t. \quad \mathbb{E}_{(s_t,a_t)\sim\rho^\pi}\left[-\log \pi_t(a_t \mid s_t)\right] \geq H \,\forall t$$

In practice: set α as a learnable parameter and perform gradient descent on the objective

$$J(\alpha) = \mathbb{E}_{a_t\sim\pi_t}\left[-\alpha \log \pi_t(a_t \mid s_t) - \alpha\bar{H}\right]$$

$[\theta, w_1, w_2, \alpha]$

The SAC algorithm

Algorithm 1 Soft Actor-Critic

Input: θ_1, θ_2, ϕ ▷ Initial parameters
$\bar{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$ ▷ Initialize target network weights
$\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay pool
for each iteration **do**
  for each environment step **do**
    $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t|\mathbf{s}_t)$ ▷ Sample action from the policy
    $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$ ▷ Sample transition from the environment
    $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$ ▷ Store the transition in the replay pool
  end for
  for each gradient step **do**
    $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
    $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ ▷ Update policy weights
    $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha J(\alpha)$ ▷ Adjust temperature
    $\bar{\theta}_i \leftarrow \tau\theta_i + (1-\tau)\bar{\theta}_i$ for $i \in \{1, 2\}$ ▷ Update target network weights
  end for
end for
Output: θ_1, θ_2, ϕ ▷ Optimized parameters

SAC experimental results

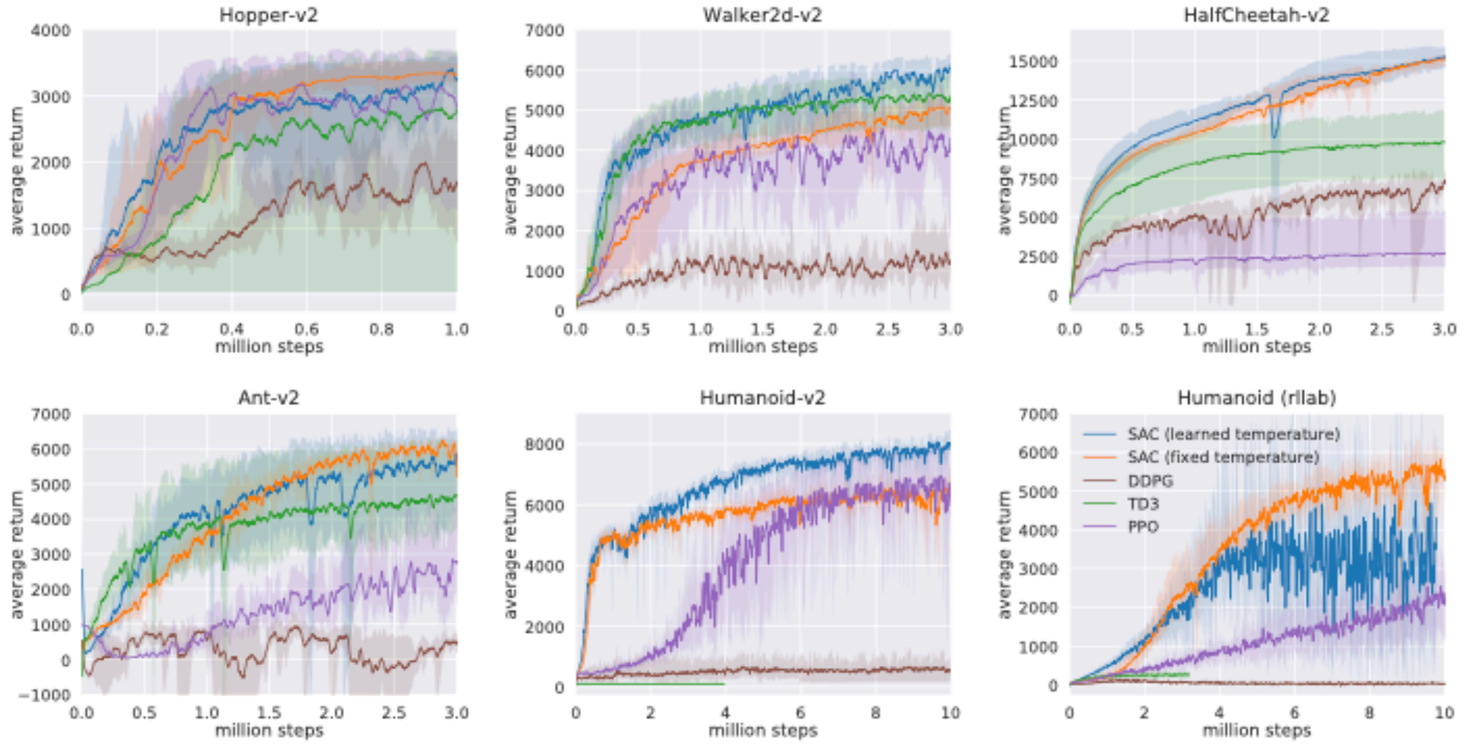


Figure 1: Training curves on continuous control benchmarks. Soft actor-critic (blue and yellow) performs consistently across all tasks and outperforming both on-policy and off-policy methods in the most challenging tasks.

SAC experimental results: efficient enough for RL on hardware

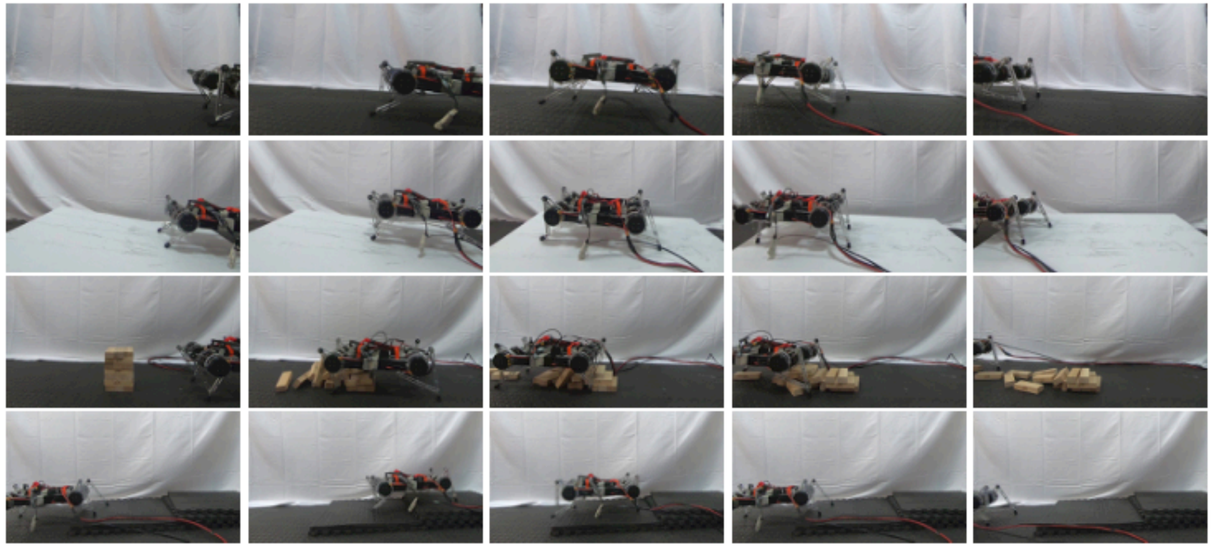


Figure 2: We trained the Minitaur robot to walk on flat terrain (first row) and tested how the learned gait generalizes to unseen situations (other rows)

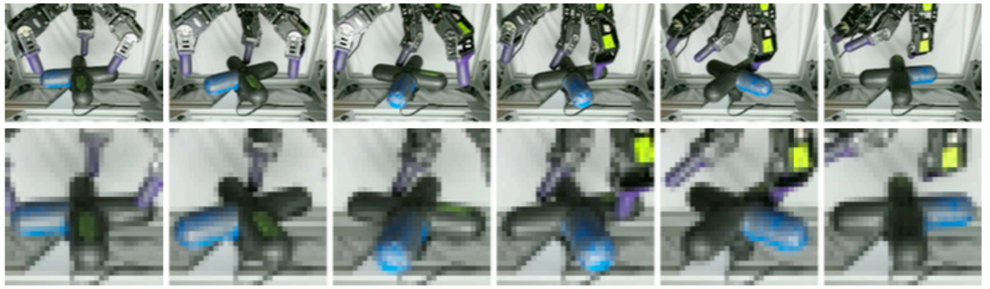
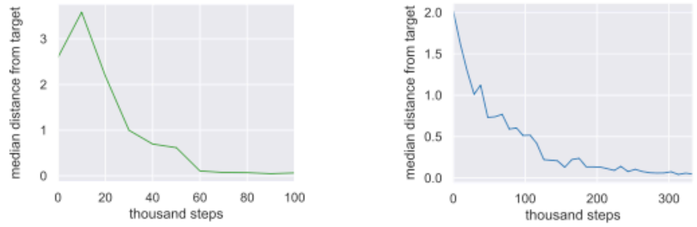


Figure 3: Valve rotation task with Dynamixel Claw. Top row shows a high resolution image sequence of a rollout with the policy. Bottom row shows the 32x32 pixel image observations of the same situations. The policy is also provided the joint positions and velocities of the fingers, but it has to infer the valve position from the image.



(a) Learning from valve joint angle.

(b) Learning from images.

Figure 4: Learning curves for the valve rotation task. The curves correspond to the median distance (measured in radians) of the valve from the target angle during a rollout. (a) Learning without images takes about 3 hours. In this case, the valve is reset to point away from the target initially. (b) Learning from images takes about 20 hours. The initial valve position is sampled uniformly at random.